

KlariVision Perde Motorları

Dört algoritma, sıfırdan anlatım

YIN v1 · Pitch Engine V2 · VPM-like · HAPT v1
ve çevrimdışı referans pYIN

Her adımın matematiği, gerekçesi, kod karşılığı
ve çalıştırılabilir Python örnekleri
Yazılıma ve sinyal işlemeye yeni başlayanlar için

31 Ağustos 2026 · kod sürümü [5a39822](#) + çalışma ağacı

İçindekiler

1. Ortak temeller — her motorun paylaştığı fikirler

Ses nedir, perde nedir · Çerçeveleme · RMS kapısı · Lag ve frekans · Sent · Üç aile: CMND, ACF, NSDF · Spektral prob

2. **YIN v1** — fark fonksiyonu, kümülatif normalizasyon

Seçim merdiveni: oktav kanıtı → hayalet cezası → süreklilik → sıçrama histerezisi → yayın kapısı · Python

3. **Pitch Engine V2** — dört tahminci, SWIPE' asal harmonik çekirdeđi

Seđim: emisyon puanı ve sabit gecikmeli Viterbi · Python

4. **VPM-like** — normalize otokorelasyon, katlardan periyot rafinesi

Seçim: kabul kuralı, spektral akraba düzeltmesi, veto durum makinesi · Python

5. **HAPT v1** — NSDF, faz kilitli anlık frekans

Seçim: harmonikler-arası veto, anlamlılık ağırlığı, iki eşikli histerezis · Python

6. **pYIN** — çevrimdışı olasılıksal referans
7. **Çevrimdışı iyileştirme katmanı** — [offline_track](#) profili · Python
8. **Karşılaştırma** — hangi motor neyi iyi yapıyor, nerede yanılıyor
9. **Ek A** — parametre tabloları
0. **Ek B** — dosya haritası
1. **Ek C** — Python örnekleri hakkında

1. Ortak temeller

Dört motor da aynı sorunu çözüyor ve aynı yapı taşlarını kullanıyor. Bu bölümü bir kez okursanız, sonraki dört bölüm sadece "bu motor bu taşları nasıl diziyo" sorusunun cevabı olur.

1.1 Ses nedir, perde nedir

Mikrofon size saniyede 44.100 (veya 22.050, 48.000) tane sayı verir. Her sayı, o andaki hava basıncıdır. Elimizde şöyle bir dizi olur:

$$x[0], x[1], x[2], \dots, x[N-1]$$

Klarnetten çıkan bir nota, bu dizide **kendini tekrar eden bir desendir**. Desen saniyede 440 kez tekrarlanıyorsa buna 440 Hz deriz ve "La" diye adlandırırız. Yani perde bulmak, şu tek soruyu cevaplamaktır:

Temel soru

Bu sayı dizisi kaç örnek sonra kendini tekrarlıyor?

O "kaç örnek" sayısına **lag** (gecikme) veya **periyot** denir; matematikte τ (tau) harfiyle gösterilir. Frekansa çevirmek tek bir bölme işlemidir:

$$f_0 = f_s / \tau$$

f_s : örnekleme hızı (Hz) · τ : periyot (örnek sayısı) · f_0 : temel frekans (Hz)

Örnek: 44.100 Hz'de $\tau = 100$ örnek ise $\rightarrow 44100 / 100 = 441$ Hz. Dört motorun tamamı, doğru τ 'yu bulmanın farklı yollarıdır. Farkları *hangi kanıta baktıkları ve ne zaman "bilmiyorum" dedikleridir*.

1.2 Çerçeveleme: sesi dilimlere bölmek

Perde zamanla değişir; tüm kaydı tek seferde analiz edemezsiniz. Ses kısa dilimlere (çerçeve / *frame* / pencere) bölünür ve her dilim için ayrı bir cevap üretilir.

- **Pencere boyu (window)**: bir dilimde kaç örnek var. Uzun pencere $\rightarrow$ frekans daha kesin ölçülür, ama hızlı değişimler (süsleme, vibrato tepesi) bulanıklaşır.
- **Adım (hop)**: bir sonraki dilime kaç örnek ilerlenir. Küçük adım $\rightarrow$ zamanda daha sık ölçüm; pencereler üst üste biner.

Üretimdeki değerler tüm motorlar için ortaktır: pencere 1536 örnek, adım 512 örnek. 44.1 kHz'de bu, 34,8 ms uzunluğunda bir pencere ve saniyede ~86 ölçüm demektir.

```
std::size_t window_size{1536};
std::size_t hop_size{512};
```

core/include/klarivision/core/analysis_engine.hpp:35–36

Bir dilimin zaman damgası pencerenin **ortasıdır**, başı değil — ölçüm o pencerenin tamamını temsil eder:

```
const double time = (static_cast<double>(start) + config.window_size / 2.0) / rate;
```

core/src/analysis_engine.cpp:1190 – `PitchEngine::analyse_causal`

1.3 RMS kapısı: sessizlikte perde aramamak

Sessiz bir pencerede perde aramanın anlamı yoktur; gürültüde rastgele bir "periyot" bulunur. Bu yüzden her motor önce yüksekliğe bakar. RMS (kök ortalama kare) sinyalin enerji ölçüsüdür:

$$RMS = \sqrt{(1/N) \cdot \sum_{n=0}^{N-1} (x[n] - \bar{x})^2}$$

$\bar{x}$ ortalama (DC ofseti); önce çıkarılır, çünkü sabit bir kayma her otokorelasyonu şişirir


```
double centered_rms(std::span<const float> samples) {
    const double mean = std::accumulate(...) / samples.size(); // DC offset
    double sum = 0;
    for (const float sample : samples) { const double x = sample - mean; sum += x * x; }
    return std::sqrt(sum / samples.size());
}
```

core/src/analysis_engine.cpp:29–37

Üretim eşiği `minimum_rms = 0.015`. Bunun altındaki pencereler hiçbir motor tarafından analiz edilmez.

1.4 Sent: müzikal mesafe birimi

İki frekansın "ne kadar uzak" olduğunu Hz cinsinden ölçmek yanıltıcıdır: 100 Hz ile 110 Hz arası kulakta büyük, 1000 Hz ile 1010 Hz arası neredeyse duyulmaz bir farktır. Müzikte mesafe logaritmiktir. Birimi **senttir**; bir oktav 1200 senttir, bir yarım ses 100 senttir.

$$\text{cents}(a, b) = 1200 \cdot \log_2(a / b)$$

```
double cents_distance(double a, double b) { return std::abs(1200.0 * std::log2(a / b)); }
```

core/src/analysis_engine.cpp:66

Bu fonksiyon dört motorda da vardır ve şu üç yerde kullanılır: süreklilik cezası, oktav sıçraması tespiti, "aynı notaya döndük mü" kontrolü.

1.5 Üç akraba fonksiyon: CMND, ACF, NSDF

Motorların üçü periyodu zaman düzleminde arar ve hepsi aynı temel hamleyi yapar: **sinyali kendisinin τ kadar kaydırılmış hâliyle karşılaştırır**. Sadece karşılaştırma biçimleri farklıdır.

a) Ham otokorelasyon (ACF) — "ne kadar benziyor?"

$$r(\tau) = \sum_j x[j] \cdot x[j+\tau]$$

Sinyal kendine benzediğinde çarpımlar aynı işaretli olur ve toplam büyür. **Tepe** ararsınız. Sorunu: sonuç ses seviyesine bağlıdır, gür çalınca büyür. Bu yüzden hiçbir motor ham ACF kullanmaz, normalize eder.

b) YIN'in fark fonksiyonu ve CMND — "ne kadar farklı?"

$$d(\tau) = \sum_j (x[j] - x[j+\tau])^2$$

Farkın karesi. Doğru periyotta sıfıra yaklaşır; **dip** ararsınız. Genlikten görece bağımsızdır. YIN v1 ve V2'nin YIN üretici bunu kullanır (Bölüm 2).

c) NSDF (McLeod) — "iki uçtan normalize edilmiş benzerlik"

$$n(\tau) = 2 \cdot \sum_j x[j] \cdot x[j+\tau] / \sum_j (x[j]^2 + x[j+\tau]^2)$$

Pay ACF'nin kendisi, payda ise karşılaştırılan iki pencerenin toplam enerjisi. Sonuç $[-1, 1]$ aralığına oturur: 1,0 "mükemmel tekrar" demektir. Genlikten tamamen bağımsızdır ve doğrudan bir güven skoru gibi okunabilir. MPM ve HAPT bunu kullanır.

```
if (normalisation > 1e-12) {
    nsdf[lag] = 2.0 * correlation / normalisation; // McLeod's NSDF normalisation
}
```

core/src/mpm.cpp:74 · aynı core/src/hapt.cpp:110

Hepsinin ortak zaafı

Bir sinyal τ periyoduyla tekrarlıyorsa, 2τ , 3τ , 4τ ile de tekrarlar — iki tur da bir tekrardır. Bu yüzden her üç fonksiyon da doğru periyodun **katlarında** da güçlü sinyal verir. Motorların karmaşıklığının büyük kısmı bu tek belirsizliği çözmek içindir: buna **oktav hatası** (veya alt-harmonik / hayalet) denir.

1.6 Parabolik interpolasyon: tam sayıdan kurtulmak

τ örnek sayıdır, yani tam sayıdır. Ama gerçek periyot 100 değil 100,37 örnek olabilir. Bu fark müzikal olarak büyüktür: 44.1 kHz'de $\tau=100 \rightarrow 441,0$ Hz, $\tau=101 \rightarrow 436,6$ Hz; arada 17 sent var, kulakla duyulur.

Çözüm: tepenin (veya dibin) etrafındaki üç noktadan bir parabol geçirip **parabolün** tepe noktasını bulmak. Türev sıfırlanınca:

$$\delta = \frac{1}{2} \cdot (y_{\tau-1} - y_{\tau+1}) / (y_{\tau-1} - 2y_{\tau} + y_{\tau+1}), \tau^* = \tau + \delta$$

pay: dibin sola mı sağa mı kaçık olduğu · payda: eğrilik (ikinci türev)

Dört motor da bunu aynı biçimde, δ 'yi $\pm 0,5$ 'e kırparak uygular (düz bir eğride payda sıfıra yaklaşır ve δ patlayabilir):

```
const auto correction = std::abs(denominator) > 1e-9
    ? std::clamp(0.5 * (previous - next) / denominator, -0.5, 0.5)
    : 0.0;
const auto refined_lag = static_cast<double>(lag) + correction;
```

core/src/mpm.cpp:101–104 · eşleri: analysis_engine.cpp:124–127, vpm_like.cpp:40, hapt.cpp:33

1.7 Spektral prob: tek frekansta "orada ses var mı?"

Zaman düzlemi periyodu bulur ama *hangi harmoniğin gerçekten çaldığını* bilmez. Bunu öğrenmek için frekans düzlemine bakmak gerekir. Tam FFT yerine motorlar çoğu zaman **tek frekanslık DFT** kullanır: sinyali tam olarak sorulan frekanstaki bir sinüs ve kosinüsle çarpıp toplarlar.

$$X(f) = \sum_n w[n] \cdot x[n] \cdot \cos(2\pi f n / f_s) - i \cdot \sum_n w[n] \cdot x[n] \cdot \sin(2\pi f n / f_s)$$

$$w[n] = 0,5 - 0,5 \cdot \cos(2\pi n / (N-1)) \quad (\text{Hann penceresi})$$

$E(f) = |X(f)|^2$ enerji, $|X(f)|$ genlik, $\arg(X(f))$ faz

Neden tam FFT değil? Çünkü aday frekanslar (441,3 Hz gibi) FFT'nin bin merkezlerine denk gelmez; tek frekanslık DFT **tam istenen frekansta** ölçüm verir. Hann penceresi ise pencerenin kenarlarını yumuşatarak spektral sızıntıyı azaltır.

```
std::complex<double> hann_windowed_probe(std::span<const float> samples, double sample_rate, double
frequency_hz)
```

core/src/harmonic_probe.cpp:8–36 — ortak yardımcı; her motorun kendi kopyası da var

Bu probun **faz** bilgisi yalnız HAPT tarafından kullanılır (Bölüm 5); diğerleri sadece genliğe/enerjiye bakar.

Pencerenin çözünürlük sınırı

Bir pencere ne kadar kısaysa, spektrumda o kadar yakın iki frekansı ayırt edemez. Hann penceresinin ana lobu 4 DFT bini genişliğindedir:

$$\text{yarı genişlik} = 2 \cdot f_s / N$$

```
return 2.0 * sample_rate / static_cast<double>(sample_count);
```

core/src/harmonic_probe.cpp:45

1536 örnek @44,1 kHz için bu ≈ 57 Hz. HAPT bu sayıyı doğrudan bir karar olarak kullanır: "bu frekans için doğrudan spektral prob güvenilir mi, yoksa zaman düzlemine mi düşeyim?" (Bölüm 5.3).

1.8 Klarnetin fiziği: neden bu iş zor

Bu belgedeki karmaşıklığın büyük kısmı tek bir fiziksel gerçekten doğuyor.

Klarnet kapalı bir borudur

Bir ucu kamışla kapalı, silindirik bir boru yalnızca **tek sayılı harmonikleri** güçlü üretir: f , $3f$, $5f$... Çift harmonikler ($2f$, $4f$) fizik gereği zayıftır. Üstelik 3. harmonik çoğu zaman temel frekanstan **daha gürdür**.

Sonuç: otokorelasyon ailesinin tamamı, temel frekansı zayıf gördüğü için sürekli olarak yanlış katlara kilitlenmeye eğilimlidir. Klarnet ayrıca oktavdan ($2\times$) değil **onikiliden ($3\times$) aşırı üfler**, yani register değişimi de $3\times$ oranındadır.

Bu yüzden dört motorda da aynı üç savunma tekrar tekrar karşınıza çıkacak:

1. **Tek harmonik doluluğu**: gerçek bir temel frekans f , $3f$ ve $5f$ 'te enerji bırakır; sahte bir alt-harmonik bırakmaz.
2. **Aşağı sıçrama histerezisi**: ani ve büyük bir aşağı sıçrama, birkaç kare boyunca onaylanmadan yayınlanmaz.
3. **Bırakış (release) tespiti**: notanın bitişi, müzikal bir kısılmadan farklıdır; RMS çok daha hızlı düşer.

2. YIN v1

Projedeki en eski motor. Adındaki "v1" geriye dönük uyumluluk içindir, kalite sıralaması değil. Fikri tek cümleyle: **sinyali kendisiyle karşılaştırıp farkın en küçük olduğu gecikmeyi bul, ama farkı adil kıyaslanabilir hâle getirmek için normalize et.** Bu bölümün ikinci yarısı asıl zor kısma ayrılmış: aday listesi elde edildikten sonra hangisinin seçileceğine nasıl karar veriliyor.

Nerede	Dosya	Ne işe yarıyor
Referans sürüm	<code>src/klarivision/pitch/yin.py</code>	Saf NumPy, 108 satır. Algoritmanın ders kitabı hâli.
Üretim (C++)	<code>core/src/analysis_engine.cpp:86</code>	<code>yin_candidates</code> + karar katmanı. JSON çıktılarıdaki <code>yin_v1</code> budur.
Canlı (Swift)	<code>LivePitchAnalyzer.swift:3772</code>	vDSP hızlandırılmış kopya, aynı formül.

2.1 Fark fonksiyonu $d(\tau)$

Fikir basit: sinyali τ kadar kaydırılmış hâliyle karşılaştır. τ gerçek periyotsa kaydırılmış hâli orijinaline çok benzer, fark ≈ 0 olur.

$$d(\tau) = \sum_{j=0}^{W-1} (x[j] - x[j+\tau])^2$$

de Cheveigné & Kawahara (2002), denklem 6

PYTHON – ADIM 1

```
# --- 2. fark fonksiyonu d(tau) ---
def fark_fonksiyonu(x, tau_min, tau_max):
    d = [0.0] * (tau_max + 1)
    for tau in range(tau_min, tau_max + 1):
        toplam = 0.0
        for j in range(len(x) - tau):
            e = x[j] - x[j + tau]
            toplam += e * e          # farkın karesi
        d[tau] = toplam
    return d
```

Python referansındaki NumPy karşılığı aynı şeyi tek satırda yapar:

```
difference = np.array(
    [np.sum((frame[:-lag] - frame[lag:])** 2) for lag in lags],
    dtype=np.float64,
)
```

`src/klarivision/pitch/yin.py:76–79`

Neden fark, neden otokorelasyon değil?

Ham otokorelasyon (Bölüm 1.5a) ses seviyesine bağlıdır: gür çalınca büyür. YIN farkın karesini alır ve minimum arar — genlikten görece bağımsız, sıfıra yaklaşan temiz bir gösterge olur.

Üretimdeki hızlandırma: aynı formül, farklı hesap

Yukarıdaki Python döngüsü her τ için pencerenin tamamını dolaşır; bu, gerçek zamanlı çalışmak için fazla yavaştır. Parantezi cebirsel olarak açarsanız:

$$d(\tau) = \sum_j x[j]^2 + \sum_j x[j+\tau]^2 - 2 \cdot \sum_j x[j] \cdot x[j+\tau]$$

ön enerji + kaydırılmış enerji – 2 × korelasyon

İlk iki terim, önceden hesaplanan **kümülatif enerji tablosundan** $O(1)$ sürede okunur; üçüncü terim tek bir nokta çarpımıdır ve Apple'ın vektör donanımında (`vDSP_dotpr`) koşar.

```
std::vector<float> energy(samples.size() + 1, 0);
for (std::size_t index = 0; index < samples.size(); ++index) {
    energy[index + 1] = energy[index] + samples[index] * samples[index];
}
...
const float correlation = float_dot(samples.data(), samples.data() + lag, count);
difference[lag] = std::max(0.0F,
    energy[count] + (energy[samples.size()] - energy[lag]) - 2.0F * correlation);
```

core/src/analysis_engine.cpp:91–106

`std::max(0.0F, ...)` şunun içindir: matematiksel olarak $d(\tau)$ negatif olamaz, ama float aritmetiğinde iki büyük sayının farkı $-0,0000001$ çıkabilir; kırılır.

Swift'teki karşılığı satır satır aynıdır ([LivePitchAnalyzer.swift:3794–3804](#)); yorumu neden böyle yapıldığını da söylüyor: eski Double/ArraySlice sürümü canlı yakalamayı ~ 9 Hz'e düşürüyormuş.

Aranan τ aralığı

$$\tau_{\min} = f_s / f_{\max}, \tau_{\max} = \min(f_s / f_{\min}, W/2)$$

$W/2$ sınırı önemli: pencerenin yarısından uzun bir periyodu ölçmek için yeterli tekrar yoktur.

Alt sınır 80 → 120 Hz

Üretim `minimum_frequency_hz` değeri 120 Hz'e çekildi. Bu bir görüntüleme tercihi değil: doğrudan $\tau_{\max}$ 'ı, yani otokorelasyonun arayabileceği en uzun periyodu kısıtlıyor. Fazla geniş bir lag aralığı, ACF'nin gerçek periyodun katına kilitlenmesine izin veren şeyin ta kendisi. Şükrü Tunar kaydında dinleyicinin işaretlediği her aşağı oktav hatası 83–160 Hz arasına düşmüş, motorların hemfikir olduğu hiçbir kare 146 Hz'in altına inmemiş. 120 Hz, sol klarinetin en pes sesini ($\sim 123,5$ Hz) açıkta bırakıyor.

core/include/klarivision/core/analysis_engine.hpp:16–32

2.2 CMND: kümülatif ortalama normalize edilmiş fark

Ham $d(\tau)$ tek başına kullanılamaz, iki tuzağı var:

1. $d(0) = 0$ 'dır (sinyal kendisiyle birebir aynıdır). En küçüğü körü körüne ararsanız hep 0'ı bulursunuz.
2. τ büyüdükçe $d(\tau)$ genelde büyür; bu karşılaştırmayı adaletsiz kılar.

YIN'in çözümü: her $d(\tau)$ 'yu, o noktaya kadarki **ortalama** d değerine böl.

$$d'(\tau) = d(\tau) / \left[(1/\tau) \cdot \sum_{j=1}^{\tau} d(j) \right] = d(\tau) \cdot \tau / \sum_{j=1}^{\tau} d(j)$$

denklem 8; $d'(0) = 1$ tanımlanır

Payda bir ortalama değildir, dolayısıyla d' birimsizdir ve tipik değeri 1 civarındadır. Gerçek periyotta $d(\tau)$ ortalamanın çok altına düştüğü için d' **sıfıra çakılır**. Artık τ 'lar adil kıyaslanabilir ve 0,15 / 0,50 gibi mutlak eşikler anlam kazanır.

PYTHON – ADIM 2

```
# --- 3. CMND: kumulatif ortalama normalize edilmiş fark ---
def cmnd(d, tau_max):
    dp = [1.0] * (tau_max + 1)      # d'(0) = 1 tanımlanır
    kosan_toplam = 0.0
    for tau in range(1, tau_max + 1):
        kosan_toplam += d[tau]
        if kosan_toplam > 0:
            dp[tau] = d[tau] * tau / kosan_toplam
    return dp
```

Üretimdeki C++ karşılığı bölme yerine çarpma kullanır — aynı şey:

```
total += difference[lag];
if (total > 0) cumulative[lag] = difference[lag] * static_cast<float>(lag) / total;
```

core/src/analysis_engine.cpp:112–116 · Python referansı: yin.py:80

Üretimde ders kitabından bir sapma

C++ ve Swift'te `difference` dizisi yalnız $[\tau_{\min}, \tau_{\max}]$ aralığında doldurulur, altındaki girdiler 0 kalır. Ama τ çarpanı hâlâ 1'den sayar. Yani payda gerçekte $\sum_{j=\tau_{\min}}^{\tau} d(j)$ iken çarpan τ 'dur; bu, ortalamayı $\tau/(\tau - \tau_{\min} + 1)$ katı küçültür ve d' olduğundan **büyük** çıkar.

Etkisi en küçük τ 'larda serttir: tam $\tau = \tau_{\min}$ noktasında $d' = \tau_{\min}$ olur (44,1 kHz / 1500 Hz için 29) — eşik 0,50'nin çok üstünde, yani en tiz birkaç lag pratikte hiç seçilemez. τ büyüdükçe çarpan 1'e yaklaşır ve fark kaybolur. Yukarıdaki Python (ve `yin.py`) tüm lag'leri doldurduğu için bu sapmayı taşımaz.

2.3 Parabolik interpolasyon ve aday toplama

τ bir tam sayıdır ama gerçek periyot 200 değil 200,37 örnek olabilir. Diben etrafındaki üç noktadan bir parabol geçirip parabolün tepe noktası alınır (Bölüm 1.6):

PYTHON – ADIM 3

```
# --- 4. parabolik duzeltme ---
def parabolik(y, i):
    sol, orta, sag = y[i-1], y[i], y[i+1]
    payda = sol - 2*orta + sag
    if abs(payda) < 1e-12:
        return 0.0
    return max(-0.5, min(0.5, 0.5 * (sol - sag) / payda))
```

Şimdi hepsini birleştiririm. Dikkat edilecek nokta: fonksiyon **tek bir cevap değil, bir liste** döndürüyor.

PYTHON – ADIM 4

```
# --- 5. aday toplama: tek cevap degil, liste ---
def yin_adaylari(x, sr, f_min=120.0, f_max=1500.0, esik=0.50):
    tau_min = max(2, int(sr / f_max))
    tau_max = min(len(x) // 2, int(sr / f_min))
    d = fark_fonksiyonu(x, tau_min, tau_max)
    dp = cmnd(d, tau_max)

    adaylar = []
    for tau in range(tau_min + 1, tau_max):
        yerel_minimum = dp[tau] <= dp[tau-1] and dp[tau] < dp[tau+1]
        if not (yerel_minimum and dp[tau] < esik):
            continue
        tau_yildiz = tau + parabolik(dp, tau)
        frekans = sr / tau_yildiz
        if f_min <= frekans <= f_max:
            adaylar.append({"frekans": frekans,
                           "guven": max(0.0, min(1.0, 1.0 - dp[tau])),
                           "lag": tau})
    adaylar.sort(key=lambda a: -a["guven"])
    return adaylar[:12] # liste 12 adayla sinirli
```

Deneyelim

Sentetik bir klarnet sesi üretelim: temel frekans 220 Hz ama **zayıf**, 3. harmonik ise **gür** — kapalı borunun tipik imzası (Bölüm 1.8).

PYTHON – SENTETİK SES

```
# --- 1. sentetik klarnet sesi ---
def klarnet(f0, sr, n):
    """Zayıf temel, guclu 3. harmonik: kapali borunun tipik imzası."""
    x = []
    for i in range(n):
        t = i / sr
        x.append(0.30 * math.sin(2*math.pi*f0*t) # temel: zayıf
                + 1.00 * math.sin(2*math.pi*3*f0*t) # 3. harmonik: gür
                + 0.45 * math.sin(2*math.pi*5*f0*t)) # 5. harmonik
    return x
```

220.00 Hz	guven 0.999	lag 200
133.38 Hz	guven 0.713	lag 331
162.98 Hz	guven 0.695	lag 271
339.13 Hz	guven 0.654	lag 130
630.94 Hz	guven 0.515	lag 70

Okunacak şeyler:

- **Doğru cevap 220 Hz, güven 0,999.** Temel frekans zayıf olmasına rağmen YIN onu buluyor — çünkü 220 Hz, 3f ve 5f'in ortak periyodudur. "Eksik temel" durumunu zaman düzlemi doğru çözer.
- **Dört yanlış aday da listede.** 133 Hz, 163 Hz, 339 Hz, 631 Hz — hepsi 0,50 eşiğinin altına düşmüş gerçek yerel minimumlar. Hiçbiri atılmıyor; karar üst katmana bırakılıyor.
- **Güvenler kendi başına yeterli değil.** Burada doğru cevap en yüksek güvene sahip; ama aşağıdaki bölümlerin tamamı, *olmadığı* durumlar için var.

2.4 Aday seçiminin iki farklı stratejisi

Aday listesi elde edildikten sonra iki farklı yaklaşım var ve proje ikisini de barındırıyor.

a) Python referansı: "eşiğin altına düşen ilk dip"

Klasik YIN tek bir cevap seçer ve bunu şöyle yapar: eşiğin (0,15) altına düşen **ilk** τ 'yu al — en küçükünü değil, ilkin.

Neden "ilk", "en derin" değil?

Çünkü gerçek periyot τ ise, 2τ ve 3τ da dip yapar: iki tur da bir tekrardır, üç tur da. Üstelik bu dipler çoğu zaman **daha derindir**, çünkü daha uzun bir aralıkta ortalama alınır. En derin dibi seçerseniz sistematik olarak bir oktav (veya bir onikili) aşağı kayarsınız. İlk dibi seçmek, "en kısa gerçek periyot" demektir ve doğru cevabı verir.

```
below_threshold = np.flatnonzero(candidate_values < settings.threshold)
if len(below_threshold):
    candidate_index = int(below_threshold[0] + minimum_lag - 1)
    while (candidate_index < len(cumulative_mean) - 1
          and cumulative_mean[candidate_index + 1] <= cumulative_mean[candidate_index]):
        candidate_index += 1
else:
    candidate_index = int(np.argmin(candidate_values) + minimum_lag - 1)
```

src/klarivision/pitch/yin.py:82-93

while döngüsü ince bir düzeltme: eşiği geçtiğiniz nokta dibin *yamacı* olabilir; iniş sürdükçe kaydırıp dibin tabanına inersiniz. Hiçbir τ eşiği geçemezse en küçük değer alınır (yedek yol).

b) Üretim: "hepsini topla, kararı ertele"

C++/Swift eşiği 0,50'ye gevşetir ve tek bir cevap seçmez. Sebebi Swift'in kendi yorumunda yazılı:

```
// Keep weaker local minima too. The continuity scorer above can
// then rescue a quiet fundamental instead of selecting its louder
// harmonic; initial/noisy frames still require high confidence.
```

macos/.../LivePitchAnalyzer.swift:3819-3821

Yani: zayıf bir aday *şu an* zayıf görünüyor olabilir, ama önceki karede izlenen kontura yakınsa doğru cevap odur. Bu bilgi aday üreticinde yoktur — bir üst katmanda vardır. O yüzden karar oraya taşınır.

Bu, dört motorun ortak tasarım tercihidir ve V2'de en uç hâline varır (Bölüm 3): kanıtı erken çökertme.

2.5 Frekans ve güven

$$f_0 = f_s / \tau^* , \text{ güven} = \text{clip}(1 - d'(\tau), 0, 1)$$

Sezgi doğrudan: d' ne kadar sıfıra yakınsa sinyal o kadar temiz periyodiktir. $d' = 0,05 \rightarrow$ güven 0,95 (kristal netliğinde nota). $d' = 0,45 \rightarrow$ güven 0,55 (şüpheli; gürültü ya da iki nota arası geçiş).

Klasik YIN burada biter. Kalan her şey — bu bölümün asıl konusu — bu ham aday listesini klarnette kullanılabilir bir karara dönüştüren seçim katmanıdır.

2.6 Seçim merdiveni

Buradan sonrası tek bir soruyu cevaplıyor: elimizde beş aday var, hangisini yayınlayacağız — ve yayınlayacak mıyız? Beş basamak sırayla uygulanır; her basamak bir öncekinin kararını değiştirebilir.

1 Doğrudan oktav kanıtı var mı? (kısa devre)

Aday üreticinin kendisi, bazı adayların yanına **0,99999** gibi neredeyse tam bir güven yazar. Bu sıradan bir CMND dibi değildir; yalnızca çok özel bir kanıt varsa üretilir: adayın $2\times$ frekansı, spektrumda kendisinden **8 kat** baskınsa **ve** tek harmonikler de üst hattı destekliorsa.

Tek harmonik koşulu neden eklendi

Ham $2\times$ baskınlığı tek başına, **gerçek** temel frekansları da yukarı terfi ettiriyordu. Sebebi Bölüm 1.8: kapalı silindirde çift kısımlar zayıftır, dolayısıyla gerçek bir temel frekans da "baskılanmış" görünür. Ayırt edici test tek harmonik doluluğudur: gerçek bir temel f , $3f$ ve $5f$ 'te enerji bırakır; f 'te hayalet olan (gerçek nota $2f$ olan) bir aday için $3f$ ve $5f$, gerçek notanın kısımları *arasındaki* boşluğa denk gelir.

Ölçüm: YIN v1'in işaretli 13 oktav hatasının **12'si** bu pastan geliyordu, hepsi 800 Hz'in üstünde.

```
const auto odd_support = [&](const double frequency) {
    double total = 0;
    for (const double multiple : {3.0, 5.0}) {
        const double probe = frequency * multiple;
        if (probe < rate / 2) total += spectral_energy(samples, rate, probe);
    }
    return total;
};
...
if (spectral_energy(samples, rate, upper) <=
    spectral_energy(samples, rate, result[index].frequency) * 8) continue; // 2x baskın mı?
if (odd_support(upper) <= odd_support(result[index].frequency)) continue; // tek harmonikler de üst hatta mı?
result.push_back({upper, 0.99999, std::log(0.99999)});
```

core/src/analysis_engine.cpp:148–171

Seçim katmanı böyle bir adayı görürse **hiçbir şeye bakmadan** onu seçer — süreklilik bile onu geri çekemez:

```
if (direct_high != candidates.end() &&
    direct_high->frequency > 800 && direct_high->confidence >= .9999) return *direct_high;
```

core/src/analysis_engine.cpp:214–215

Gerekçe: bu aday zaten doğrudan spektral kanıtla var. Sürekliliğin onu kararlı bir alt-periyoda geri çekmesine izin vermek, kanıtı yok saymak olurdu.

2 Hayalet alt-harmonik cezası

Şimdi ters tuzak. YIN, gerçekte var olmayan bir alt-periyoda kilitlenebilir: dalga biçimi orada tekrarlıyor gibi görünür ama o frekansta **gerçek bir ses yoktur**. Buna otokorelasyon hayaleti denir.

Test doğrudan: adayın kendi frekansında spektrumda ne kadar enerji var, $2\times$ ve $3\times$ katlarında ne kadar var?

$$r = \max_{k \in \{2,3\}} [|X(k \cdot f)| / |X(f)|], \text{ hayalet} \Leftrightarrow r > 6$$

$$\text{ceza} = 0,15 \cdot \text{clip}((r - 6) / 20, 0, 1)$$

PYTHON – HAYALET TESTİ VE CEZASI


```
# --- 1. hayalet cezası -----
def hayalet_orani(spektrum, f):
    """spektrum: frekans -> genlik sozlugu (gercekte tek frekans DFT proble olcudur)"""
    kendi = spektrum.get(f, 0.0)
    en_baskin = 0.0
    for kat in (2.0, 3.0):
        katin_genligi = spektrum.get(f * kat, 0.0)
        if katin_genligi <= 0:
            continue
        oran = katin_genligi / max(kendi, 1e-9)
        en_baskin = max(en_baskin, oran)
    return en_baskin

def hayalet_cezasi(spektrum, f, esik=6.0):
    oran = hayalet_orani(spektrum, f)
    if oran <= esik:
        return 0.0
    siddet = max(0.0, min(1.0, (oran - 6.0) / 20.0))
    return 0.15 * siddet
```

```
const double severity = std::clamp((existence.dominant_multiple_ratio - 6.0) / 20.0, 0.0, 1.0);
return 0.15 * severity;
```

core/src/analysis_engine.cpp:185-191 · eşik 6,0: core/src/harmonic_arbitration.cpp:74

6,0 eşiği nereden geliyor

Uydurulmuş değil, iki ölçülmüş kümenin arasına konmuş. Zayıf **ama gerçek** bir klarnet temel frekansı, kendi üçüncü harmoniğinin ~3 katı gerisinde kalabiliyor. Gerçek bir hayalet ise en belirsiz/geçici karesinde bile ~8 katına, temiz sürdürülen hâlinde 24-27 katına ulaşıyor. Eşik iki kümenin arasında, "gerçek ama zayıf" tarafa yakın duruyor — yani şüphede kalınca gerçek notayı korumayı tercih ediyor.

core/src/harmonic_arbitration.cpp:70-77 (gerekçe yorumda)

Cezanın **kademeli** olması bilinçli: sert bir ret, gerçekten belirsiz kareleri de silerdi. Kod yorumunun ifadesiyle, ceza "sürekliliği ve gerçekten belirsiz adayları adil biçimde yenmeli, ezip geçmemeli".

3 Süreklilik puanı — ve iki farklı yol

Müzik sıçramaz. Bir önceki karede 440 Hz duyduysanız, bu karede 880 Hz'den çok 442 Hz olması muhtemeldir. Ama bu bilgi ancak **bir önceki kare varsa** kullanılabilir. Kod bu yüzden iki ayrı yol tutar.

Yol A — önceki kare yok (nota başlangıcı)

Bu, artikülasyon boşluğundan hemen sonraki durumdur: tam da zayıf temelli / güçlü üçüncü harmonikli bir notanın otokorelasyona hayalet alt-harmonik sunma olasılığının en yüksek olduğu an. Dayanak olmadığı için baraj yükseltilir ve sıralama **ham güvene değil, hayalet-cezalı güvene** göre yapılır:

$$\text{seçim} = \operatorname{argmax}_{\text{aday}} [\text{güven} - \text{hayalet_cezası}] , \text{yalnız güven} \geq 0,76 \text{ ise}$$

Yol B — önceki kare var (normal izleme)

$$\text{skor} = \text{güven} - \text{hayalet_cezası} - 0,30 \cdot \min(\text{sent_uzaklık} / 700, 1)$$

yalnız kendi güveni $\geq 0,55$ olan adaylar yarışır

Ceza doğrusaldır ve 700 sentte tavana vurur: 700 senti aşan her sıçrama aynı -0,30'u öder, daha fazlasını değil. Bu, gerçek büyük atlamaların tümüyle imkânsız hâle gelmesini engeller.

PYTHON – İKİ YOLLU SEÇİM

```
# --- 2. nedensel secim -----
def nedensel_secim(adaylar, onceki, spektrum):
    if not adaylar:
        return None
    def duzeltilmis(a):
        return a["guven"] - hayalet_cezasi(spektrum, a["frekans"])

    if onceki is None:
        en_yyi = max(adaylar, key=duzeltilmis)
        secim = en_yyi if en_yyi["guven"] >= 0.76 else None
    else:
        secim, en_yyi_skor = None, float("-inf")
        for a in adaylar:
            if a["guven"] < 0.55:
                continue
            skor = duzeltilmis(a) - 0.30 * min(sent(a["frekans"], onceki["frekans"]) / 700.0, 1.0)
            if skor > en_yyi_skor:
                secim, en_yyi_skor = a, skor

    if secim is not None and hayalet_orani(spektrum, secim["frekans"]) > 6.0:
        return None
    return secim
```

Son iki satıra dikkat: seçilen aday *yine de* hayalet çıkarsa, hiçbir şey yayınlanmaz. Gerekçe kodda yazılı ve öğretici:

Neden hayalet asla yayınlanmaz

Klarnetin en ince, en belirsiz geçiş karelerinde merdivende **yalnızca** hayaletler kalabilir — hiçbir aday kendi akustik enerjisiyle güven barajını geçemez. Bir hayaletin, komşularına göre ne kadar iyi sıralandığından bağımsız olarak, temel frekans diye yayınlanmaya **periyot düzleminde hiçbir hakkı yoktur**. Sessiz kalmak, boşluk köprülemesinin (2.8) o kareyi çevresinden kurtarmasına izin verir.

core/src/analysis_engine.cpp:259–270

Deneyelim: yüksek güvenli hayalet vs. düşük güvenli gerçek

Kritik senaryo şu: 220 Hz adayının güveni **daha yüksek** (0,94), 660 Hz'inki daha düşük (0,88). Ama spektrumda 220 Hz'de neredeyse hiç enerji yok, 660 Hz gürül gürül. Ham güvene bakan bir seçici yanıltırdı.

PYTHON – DENEME

```
spektrum = {220.0: 0.02, 440.0: 0.03, 660.0: 0.90, 1100.0: 0.40}
adaylar = [{"frekans": 220.0, "guven": 0.94}, {"frekans": 660.0, "guven": 0.88}]

print("hayalet orani 220 Hz :", round(hayalet_orani(spektrum, 220.0), 1))
print("ceza 220 Hz          :", round(hayalet_cezasi(spektrum, 220.0), 3))
print("onceki yok  ->", nedensel_secim(adaylar, None, spektrum))
print("onceki 660  ->", nedensel_secim(adaylar, {"frekans": 660.0}, spektrum))
```

ÇIKTI

```
hayalet orani 220 Hz : 45.0
ceza 220 Hz          : 0.15
onceki yok  -> {'frekans': 660.0, 'guven': 0.88}
onceki 660  -> {'frekans': 660.0, 'guven': 0.88}
```

220 Hz'in 3× katı (660 Hz) kendisinden 45 kat gür: eşik 6'nın çok üstünde, ceza tavana (0,15) vuruyor. Ama asıl belirleyici olan ceza bile değil — son kontrol 220 Hz'i hayalet olarak işaretlediği için, **seçilseydi bile** yayınlanmayacaktı. Her iki yolda da doğru cevap kazanıyor.

4 Aşağı sıçrama histerezi

Seçim yapıldı; hâlâ bir savunma var. Son yayınlanan perdenin **900 sentten fazla altına** düşen bir aday tek karede kabul edilmez. Üç ardışık kare, her biri bekleyen adayın 360 senti içinde, aynı düşüşü onaylamalıdır.

Neden yalnız aşağı?

Çünkü otokorelasyon ailesinin hata modu asimimetrik. Yanlış kilitleme daima gerçek periyodun bir **katına** olur — yani frekansta **aşağı** doğru ($f/2$, $f/3$). Yukarı doğru bir hata için sinyalin gerçekte olmayan bir alt-periyodu olması gerekirdi. O yüzden yukarı sıçramalar anında kabul edilir, aşağı sıçramalar sorgulanır.

PYTHON – HISTEREZIS DURUM MAKİNESİ

```
# --- 3. asagi sicrama histerezisi -----
class SicramaMuhafizi:
    def __init__(self, esik_sent=900.0, gereken_onay=3, tolerans_sent=360.0):
        self.esik_sent, self.gereken_onay, self.tolerans = esik_sent, gereken_onay, tolerans_sent
        self.bekleyen, self.onay = None, 0

    def gecir(self, secim, son_yayin):
        if secim is None or son_yayin is None:
            self.bekleyen, self.onay = None, 0
            return secim
        asagi = secim["frekans"] < son_yayin["frekans"]
        if not (asagi and sent(secim["frekans"], son_yayin["frekans"]) > self.esik_sent):
            self.bekleyen, self.onay = None, 0
            return secim
        if self.bekleyen and sent(secim["frekans"], self.bekleyen["frekans"]) < self.tolerans:
            self.onay += 1
            self.bekleyen = secim
            if self.onay < self.gereken_onay:
                return None
            self.bekleyen, self.onay = None, 0
            return secim
        self.bekleyen, self.onay = secim, 1
        return None
```

```
if (impl_>pending_yin_confirmations < 3) return {}; // henüz yeterince onaylanmadı
```

core/src/analysis_engine.cpp:558-573

PYTHON – DENEME: 660 HZ İZLENİRKEN 330 HZ'E DÜŞÜŞ

```
m = SicramaMuhafizi()
son = {"frekans": 660.0}
for i, f in enumerate([330.0, 330.0, 331.0, 329.0], 1):
    cikti = m.gecir({"frekans": f, "guven": 0.9}, son)
    if cikti:
        son = cikti
        # yayinlandi: yeni sureklilik capasi
        print(f"kare {i}: aday {f:6.1f} Hz -> YAYIN")
    else:
        print(f"kare {i}: aday {f:6.1f} Hz -> beklet (onay {m.onay}/3)")
```

ÇIKTI

```
kare 1: aday 330.0 Hz -> beklet (onay 1/3)
kare 2: aday 330.0 Hz -> beklet (onay 2/3)
kare 3: aday 331.0 Hz -> YAYIN
kare 4: aday 329.0 Hz -> YAYIN
```

İlk iki kare bastırıldı; üçüncüde düşüş onaylandı ve yayınlandı. Dördüncü kare artık bir sıçrama değil — çünkü süreklilik çapası 331 Hz'e taşındı. Tek karelik bir hata bu merdivende hiç görünmez; gerçek bir register değişimi 35 ms gecikmeyle görünür.

5 Yayın kapısı: bırakış ve yavaş sönüm

Son basamak seçimle değil, **yayınlamakla** ilgili. Bir notanın bitişi müzikal bir *decrescendo*'dan farklıdır: RMS çok daha hızlı düşer. İki ardışık %20'lik düşüş bırakış tetikler ve süreklilik durumunu tamamen temizler.

```
if (impl_>previous_yin_rms > 0 && rms < impl_>previous_yin_rms * .80) ++impl_>yin_release_falls;
...
if (impl_>yin_release_falls >= 2) impl_>yin_release_active = true;
if (impl_>yin_release_active && rms >= impl_>yin_release_rms_peak * .40) {
    impl_>yin_release_active = false; // sinyal toparlandı: yanlış alarm
    impl_>yin_release_falls = 0;
}
```

core/src/analysis_engine.cpp:462-473

İki ayrı RMS tepesi tutulur ve ikisi farklı işler yapar:

Tepe	Sönüm	Ne için
<code>recent_yin_rms_peak</code>	× 0,85	Hızlı söner. "Şu an son tepenin %35'inin altında mıyız?" — yavaş fade kapısı.
<code>yin_release_rms_peak</code>	× 0,995	Yavaş söner, notanın gerçek zirvesini uzun süre hatırlar. Bırakıştan çıkış eşiği (%40) buna göre ölçülür.

Ve nihai kapı: güveni 0,90'ın altında **ve** RMS'i kendi son tepesinin %35'inin altında olan bir aday yayınlanmaz — bu bileşim "sönen reverb kuyruğu, gerçek nota değil" demektir.

```
if (choice && !impl->yin_release_active && !(choice->confidence < .90 &&
    rms < impl->recent_yin_rms_peak * .35)) {
    publication = EngineFrame{source_time, choice->frequency, choice->confidence};
}
```

core/src/analysis_engine.cpp:582-584

Dikkat: koşul bir **ve**'dir. Yalnız güvenin düşük olması yayını engellemez; yalnız sesin kısılması da engellemez. İkisi birlikte olmalıdır — çünkü gerçek bir *diminuendo*'da ses kısılır ama periyodiklik yüksek kalır.

2.7 30 kare kuralı

Arka planda sessiz bir sayaç işler: art arda 30 kare hiçbir şey yayınlanmadıysa (≈350 ms), izlenen kontur artık anlamsızdır ve tüm süreklilik durumu sıfırlanır.

```
if (impl->silent_yin_estimates >= 30) {
    impl->last_yin.reset();
    impl->pending_yin_jump.reset();
    impl->pending_yin_confirmations = 0;
    impl->pending_gap.clear();
}
```

core/src/analysis_engine.cpp:590-597

Bu olmadan, bir susmadan sonraki ilk nota, **yarım saniye önceki** notaya olan sent uzaklığıyla cezalandırılırdı. Süreklilik ancak bir süre için kanıttır.

2.8 Boşluk köprüleme

Kısa kopmalar (en fazla 7 kare) son bilinen perde tekrarlanarak doldurulur — ama **sadece** boşluğun iki ucu da yeterince güvenliyse ve birbirine 90 sentten yakınsa. Köprü, "aynı notaya geri döndük" kanıtıyla açılır.

```
std::max(last->confidence, frame->confidence) >= minimum_endpoint_confidence &&
std::min(last->confidence, frame->confidence) >= weaker_floor &&
cents_distance(*last->frequency_hz, *frame->frequency_hz) <= 90
```

core/src/analysis_engine.cpp:301-305 — `append_bridged`, `yin_v1` ve `hapt_v1` ortak

Önemli ayrıntı: köprü kareleri **anında yayınlanmaz**, tamponlanır. Ancak boşluğun öbür ucu gelip aynı perdeyi doğruladığında topluca serbest bırakılır. Doğrulama gelmezse tampon çöpe gider — yani motor asla "belki devam etmiştir" diye tahmin yayınlanmaz.

2.9 Bir karenin tam yolculuğu

1536 örnek

↓ RMS < 0,015 ? → sessizlik, çık	analysis_engine.cpp:29
↓ DC çıkar	
↓ $d(\tau) = \sum (x[j] - x[j+\tau])^2$ her $\tau \in [\tau_{\min}, \tau_{\max}]$	analysis_engine.cpp:102
↓ $d'(\tau) = d(\tau) \cdot \tau / \sum d(j)$	analysis_engine.cpp:116
↓ yerel minimumlar, $d' < 0,50$ (≤ 12 aday)	analysis_engine.cpp:122
↓ parabolik düzeltme δ	analysis_engine.cpp:124
↓ $f_0 = f_s / (\tau + \delta)$, güven = $1 - d'(\tau)$	analysis_engine.cpp:130
↓ — seçim merdiveni —	
↓ 1. doğrudan oktav kanıtı ($2 \times 8 \times$ + tek harmonik)	analysis_engine.cpp:148
↓ 2. hayalet cezası ($r > 6$)	analysis_engine.cpp:185
↓ 3. süreklilik: taze 0,76 / izleme 0,55 + 0,30·sent	analysis_engine.cpp:238, 249
↓ 4. aşağı sıçrama: 3 kare onay (> 900 sent)	analysis_engine.cpp:558
↓ 5. bırakış / yavaş-fade kapısı	analysis_engine.cpp:582
↓ boşluk köprüleme (≤ 7 kare, ≤ 90 sent, tampondan)	analysis_engine.cpp:274

EngineFrame{ zaman, frekans, güven }

Elle takip etmek isterseniz

İki başlangıç noktası: `src/klarivision/pitch/yin.py` (108 satır, dallanma yok — `_estimate_frame` 'e tek pencere verip $d(\tau)$ ve $d'(\tau)$ eğrilerini yazdırabilirsiniz) ve bu bölümdeki Python örneklerinin tamamını içeren `docs/perde-motorlari-ornekler.py`.

3. Pitch Engine V2

YIN v1'in temel varsayımı şudur: "bir tahminci çalıştır, kararı hemen ver." V2 bu varsayımın ikisini de reddeder. **Dört bağımsız tahminci** çalıştırır, hiçbirinin kararını erken kabul etmez, ve son kararı **beş kare geriden**, geleceğe bakarak verir. Bu bölümün ağırlık merkezi seçimdir — çünkü V2'de aday üretimi kolay kısım, seçim ise motorun tamamı.

3.1 Temel fikir: kanıtı erken çökertme

YIN v1'de her karede tek bir cevap seçilir ve o cevap sonraki karenin süreklilik dayanağı olur. Bir kare yanlış giderse, hata sürekliliğe sinip yayılabilir. Buna *erken çökertme* (premature collapse) diyelim: elinizde birden çok olasılık varken tek birine indirgemek, ve indirgeme kararını en az bilgiye sahip olduğunuz anda vermek.

V2 bunun yerine her karede bir **aday listesi** tutar ve her adayın nereden geldiğini hatırlar:

```
enum class CandidateSource { yin, autocorrelation, mpm, spectral };
```

core/include/klarivision/core/pitch_engine_v2.hpp:11–16 – "V2 keeps this provenance instead of collapsing every observation into one early YIN decision"

Provenance (kaynak bilgisi) sadece kayıt tutmak için değil: **uzlaşma** teriminin hesaplanabilmesi için gerekli (3.4). İki farklı tahmincinin aynı frekansı söylemesi ile aynı tahmincinin iki adayının yan yana durması, kanıt olarak taban tabana zıttır.

3.2 Dört aday kaynağı

a) YIN üretici

Bölüm 2'deki CMND algoritmasının aynısı. Farkı: periyodikliği 0,50'nin altındaki adaylar baştan atılır — çünkü V2'de her adayın SWIPE' ile çapraz kontrolü pahalıdır, listeyi budamak gerekir.

```
candidates.erase(std::remove_if(candidates.begin(), candidates.end(), [](const auto& item) {
    return item.periodicity < 0.50;
}), candidates.end());
```

core/src/pitch_engine_v2_session.cpp:311–314

b) Normalize otokorelasyon

Klasik ACF, enerjiyle normalize edilmiş, ve **tek** bir aday döndürür. Seçim stratejisi Bölüm 4'teki VPM-like ile aynıdır: en güçlü tepeyi değil, en güçlüye "yeterince yakın" olan **en kısa lag**'i seç.

core/src/pitch_engine_v2_session.cpp:146–208 – `autocorrelation_candidate`

c) MPM (McLeod Pitch Method) — NSDF

Bölüm 1.5c'deki NSDF. Her yerel maksimum bir adaydır ve hepsi döndürülür (en fazla 12).

$$n(\tau) = 2 \cdot \sum x[j] \cdot x[j+\tau] / \sum (x[j]^2 + x[j+\tau]^2)$$

core/src/mpm.cpp:9–133 (referans) · core/src/pitch_engine_v2_session.cpp:215 (vDSP'li eş sürüm)

d) Spektral oktav adayı

Mevcut adayların 2x frekansında enerji 8 kat baskınsa, o frekans `spectral` kaynaklı yeni bir aday olarak eklenir — mevcut adayın **üzerine yazılmaz**, yanına konur. Bu ayırım önemli: iki hipotez de yarışmaya devam eder.

```
if (spectral_energy(samples, sample_rate, upper) >
    spectral_energy(samples, sample_rate, candidates[index].frequency_hz) * 8) {
    candidates.push_back(PitchCandidate{
        upper, std::max(0.92, candidates[index].periodicity), 0, CandidateSource::spectral,
    });
}
```

core/src/pitch_engine_v2_session.cpp:344–350

3.3 SWIPE' harmonik desteği

Her aday, tam FFT üzerinden hesaplanan **SWIPE' benzeri spektral çekirdek** ile sınanır. Bu, V2'nin diğer motorlarda olmayan en özgün kanıt kaynağıdır.

Fikir: gerçek bir temel frekans f ise, spektrumda f , $2f$, $3f$... noktalarında **tepelere**, aralarında **çukurlar** olmalıdır. SWIPE' bunu bir iç çarpım olarak ölçer:

$$\text{skor}(f) = \sum_k w_k \cdot [S(kf) - \frac{1}{2}(S((k-\frac{1}{2})f) + S((k+\frac{1}{2})f))] / \sqrt{(\sum w_k^2 \cdot \sum S(kf)^2)}$$
$$w_k = 1/\sqrt{k}$$

$S(\cdot)$: Hann pencere FFT'nin karekök-genlik spektrumu, binler arası doğrusal interpolasyonla okunur

Üç tasarım kararı önemli:

- **Tepe eksi komşu çukurlar.** Yalnız tepe yüksekliğine bakmak, gürültülü/yassı bir spektrum bölgesini de yüksek puanlardı. Fark almak, dar ve gerçek bir harmonik tepesini ödüllendirir.
- **Sadece 1. ve asal harmonikler pozitif kanıt sayılır.** Sebep: asal olmayan bir harmonik ($4f$, $6f$, $9f$...) aynı zamanda *başka*, daha pes bir temel frekansın da harmoniğidir — $6f$, $2f$ 'in ve $3f$ 'in de harmoniğidir. Asal olanlar bu karışıklığa çok daha az düşer. Bu, bir alt-harmonik adayın yüksek puan almasını zorlaştıran doğrudan bir önlemdir.
- **Payda tüm harmonikleri sayar.** Böylece daha gür veya daha kalabalık bir spektrum otomatik olarak yüksek puan almaz — skor bir kosinüs benzerliği gibi $[0, 1]$ aralığında kalır.

```
if (harmonic != 1 && !is_prime(harmonic)) {
    continue; // only the fundamental (harmonic 1) and prime harmonics contribute positive evidence
}
const auto weight = 1.0 / std::sqrt(static_cast<double>(harmonic));
...
inner_product += weight * (peak - 0.5 * (left_valley + right_valley));
```

core/src/swipe_prime.cpp:142–151 · asallık testi: swipe_prime.cpp:15–25

Bütün adaylar tek bir FFT çağrısında toplu puanlanır ([swipe_prime_harmonic_supports](#)); FFT, dosya içinde yazılmış radix-2 Cooley–Tukey uygulamasıdır ([swipe_prime.cpp:44–100](#)).

3.4 Seçim, birinci yarı: emisyon puanı

Artık her adayın üç bağımsız kanıtı var. Bunlar tek bir sayıya indirilir — Viterbi'nin dilinde buna **emisyon** denir: "bu kare, tek başına, bu adayı ne kadar destekliyor?"

$$\text{emisyon} = 0,40 \cdot \text{periyodiklik} + 0,20 \cdot \text{uzlaşma} + 0,40 \cdot \text{SWIPE'}_{\text{desteği}}$$

$\text{uzlaşma} = 1,0$ (başka bir *kaynak* 55 sent içinde aynı frekansı söylediye), aksi hâlde $0,25$

PYTHON – EMİSYON PUANI

```
# --- 1. emisyon puanı -----
def emisyon_puanlari(adaylar):
    """adaylar: [{'frekans', 'periyodiklik', 'kaynak', 'swipe'}]"""
    sonuc = []
    for a in adaylar:
        uzlasti = any(b["kaynak"] != a["kaynak"] and sent(b["frekans"], a["frekans"]) <= 55
                     for b in adaylar)
        uzlasma = 1.0 if uzlasti else 0.25
        puan = 0.40 * a["periyodiklik"] + 0.20 * uzlasma + 0.40 * a["swipe"]
        if a["kaynak"] == "spektral":
            puan = max(puan, 0.90)
        sonuc.append({**a, "uzlasti": uzlasti, "emisyon": puan})
    return sonuc
```

```
const double agreement = agrees ? 1.0 : 0.25;
double score = 0.40 * candidates[index].periodicity + 0.20 * agreement +
    0.40 * supports[index];
```

core/src/pitch_engine_v2_session.cpp:370–380

Uzlaşma neden bu kadar değerli

YIN, ACF ve MPM aynı sinyale bakar ama farklı matematik kullanır: biri farkı minimize eder, biri korelasyonu maksimize eder, biri iki uçtan normalize eder. **Hata modları farklıdır.** Bu yüzden ikisinin 55 sent içinde aynı yeri göstermesi, tek bir tahmincinin 0,99 özgüveninden daha güçlü kanıttır — özgüven ortak bir yanılgıyı da yansıtabilir, uzlaşma yansıtamaz.

Uzlaşma yoksa terim 0 değil **0,25**'tir: "yalnız kalmak" bir suç değil, sadece bir avantajın yokluğudur.

Spektral adayın taban puanı

Spektral kaynaklı aday için puan 0,90'a **tabanlanır**:

```
if (candidates[index].source == CandidateSource::spectral) {
    score = std::max(score, 0.90);
}
```

core/src/pitch_engine_v2_session.cpp:387

Gerekçe: o aday zaten yalnızca 8× doğrudan spektral kanıtla var. Periyot düzlemindeki üç tahmincinin *alt harmonikte* anlaşması onu ezmemelidir — çünkü hepsi aynı zayıflığı paylaşır (Bölüm 1.5 kutusu). Burada uzlaşma terimi yanıltıcı olurdu ve taban bunu telafi eder.

Yüksek register barajı

900 Hz'in üstünde oktav/harmonik karışıklıkları en sık görülür. Oraya ayrı bir kanıt barajı konur:

```
return candidate.agreement_support || (candidate.frequency_hz > 900 &&
    candidate.periodicity >= 0.80 && candidate.harmonic_support >= 0.75);
```

core/src/pitch_engine_v2_session.cpp:399–400

Yani 900 Hz üstü bir aday ya **uzlaşmalı**, ya da hem periyodikliği hem spektral desteği tek başına yüksek olmalı. Bu bir yayın kapısıdır, emisyon puanının parçası değil — Viterbi seçtikten *sonra* uygulanır.

3.5 Seçim, ikinci yarı: sabit gecikmeli Viterbi

Burası V2'yi diğer üç motordan gerçekten ayıran yerdir. Diğerleri **tamamen nedenseldir**: her kareyi görür görmez karar verirler. V2 kararı **5 kare geciktirir** ve o beş karenin kanıtını da hesaba katarak en iyi *yolu* seçer.

Viterbi nedir?

Elinizde her kare için birkaç aday var. Bir "yol", her kareden bir aday seçmektir. Her yolun puanı iki parçadan oluşur:

$$\text{yol_puanı} = \sum_{\text{kare}} \text{emisyon}(\text{seçilen}) + \sum_{\text{kare}} \text{geçiş}(\text{önceki seçilen} \rightarrow \text{bu seçilen})$$

Emisyon "bu aday kendi başına ne kadar inandırıcı", **geçiş** ise "bu hareket müzikal olarak ne kadar makul" demektir. Kaba kuvvetle tüm yolları denemek üstel maliyetlidir (her karede k aday, n kare $\rightarrow k^n$ yol); Viterbi bunu dinamik programlamayla **$k^2 \cdot n$** 'ye indirir. Sır şu: bir adaya varan en iyi yolun puanı, o adaya *nereden* gelindiğine bakılarak tek adımda hesaplanabilir; geçmişin geri kalanını hatırlamaya gerek yoktur.

$$\text{geçiş}(a \rightarrow b) = -0,18 \cdot \min(|\text{cents}(a,b)| / 700, 1) - 0,12 \text{ [eğer } |\text{cents}| \geq 850]$$

ikinci terim: bir karelik oktav/onikili sıçramasına ek ücret

PYTHON – GEÇİŞ CEZASI

```
# --- 2. gecis cezasi -----
def gecis(a_hz, b_hz, genislik=700.0):
    c = sent(a_hz, b_hz)
    ceza = 0.18 * min(c / genislik, 1.0)
    if c >= 850.0:
        ceza += 0.12
    return -ceza
```

core/src/fixed_lag_tracker.cpp:25–33

Ek ücretin kasten **ölçülü** tutulduğunu kod yorumu söylüyor: sürdürülen bir register değişimi, sonraki beş karenin emisyon kanıtıyla yine de kazanabilmeli. Ceza bir yasak değil, bir fiyattır.

Dinamik programın kendisi

PYTHON – VITERBI VE SABİT GECİKMELİ İZLEYİCİ

```
# --- 3. sabit gecikmeli Viterbi -----
def viterbi_coz(kareler):
    """kareler: her biri emisyonlu aday listesi. En eski karenin kararini dondurur."""
    onceki = [a["emisyon"] for a in kareler[0]]
    geri = []
    for k in range(1, len(kareler)):
        simdi = [float("-inf")] * len(kareler[k])
        isaret = [0] * len(kareler[k])
        for i, aday in enumerate(kareler[k]):
            for j, evvel in enumerate(kareler[k-1]):
                puan = onceki[j] + gecis(evvel["frekans"], aday["frekans"])
                if puan > simdi[i]:
                    simdi[i], isaret[i] = puan, j
            simdi[i] += aday["emisyon"]
        onceki, _ = simdi, geri.append(isaret)
    secili = max(range(len(onceki)), key=lambda i: onceki[i])
    for isaret in reversed(geri):
        secili = isaret[secili]
    return kareler[0][secili]

class SabitGecikmeliIzleyici:
    def __init__(self, gecikme=5):
        self.gecikme, self.tampon = gecikme, []
    def it(self, adaylar):
        self.tampon.append(adaylar)
        if len(self.tampon) <= self.gecikme:
            return None
        karar = viterbi_coz(self.tampon)
        self.tampon.pop(0)
        return karar
    def bitir(self):
        while self.tampon:
            yield viterbi_coz(self.tampon)
            self.tampon.pop(0)
```

Üç satır özellikle dikkat çekici:

- `onceki = [a["emisyon"] for a in kareler[0]]` — arama **en öndeki kareden başlar**, daha önce yayınlanmış kareye bağlanmaz. Bunun sonucu 3.6'da.
- `geri.append(isaret)` — her adım için "en iyi yol buraya nereden geldi" saklanır; sonda bu işaretler geriye yürünerek **ilk karenin** kararı bulunur.
- `self.tampon.pop(0)` — yalnız en eski kare karara bağlanır ve tampondan çıkar; kalan dört kare bir sonraki turda *daha çok* ileri-bakışla yeniden değerlendirilir.

Deneyelim: tek karelik oktav hatası

440 Hz'lik bir nota çalıyor. Bir karede — sadece birinde — 220 Hz adayı hem uzlaşma kazanıyor hem yüksek periyodiklik gösteriyor, 440 Hz ise çöküyor. Nedensel bir seçici bu kareyi kaybeder.

PYTHON – DENEME

```

def saglam_kare():
    """440 Hz iki tahminci tarafından da bulunmus: uzlasma var."""
    return emisyon_puanlari([
        {"frekans": 440.0, "periyodiklik": 0.95, "kaynak": "yin", "swipe": 0.90},
        {"frekans": 440.0, "periyodiklik": 0.93, "kaynak": "mpm", "swipe": 0.90},
        {"frekans": 220.0, "periyodiklik": 0.60, "kaynak": "yin", "swipe": 0.30},
    ])

def bozuk_kare():
    """Tek kare: 220 Hz uzlasma da kazanıyor, 440 Hz cokuyor."""
    return emisyon_puanlari([
        {"frekans": 220.0, "periyodiklik": 0.75, "kaynak": "yin", "swipe": 0.55},
        {"frekans": 220.0, "periyodiklik": 0.74, "kaynak": "mpm", "swipe": 0.55},
        {"frekans": 440.0, "periyodiklik": 0.62, "kaynak": "yin", "swipe": 0.55},
    ])

dizi = [saglam_kare(), saglam_kare(), bozuk_kare(),
        saglam_kare(), saglam_kare(), saglam_kare()]

print("--- nedensel (kare basina en yuksek emisyon) ---")
for i, k in enumerate(dizi, 1):
    en = max(k, key=lambda a: a["emisyon"])
    print(f" kare {i}: {en['frekans']:6.1f} Hz (emisyon {en['emisyon']:.2f})")

print("--- 5 kare gecikmeli Viterbi ---")
iz = SabitGecikmeliIzleyici(gecikme=5)
cikti = [iz.it(k) for k in dizi]
cikti = [c for c in cikti if c] + list(iz.bitir())
for i, c in enumerate(cikti, 1):
    print(f" kare {i}: {c['frekans']:6.1f} Hz")

```

ÇIKTI

```

--- nedensel (kare basina en yuksek emisyon) ---
kare 1:  440.0 Hz (emisyon 0.94)
kare 2:  440.0 Hz (emisyon 0.94)
kare 3:  220.0 Hz (emisyon 0.72)
kare 4:  440.0 Hz (emisyon 0.94)
kare 5:  440.0 Hz (emisyon 0.94)
kare 6:  440.0 Hz (emisyon 0.94)
--- 5 kare gecikmeli Viterbi ---
kare 1:  440.0 Hz
kare 2:  440.0 Hz
kare 3:  440.0 Hz
kare 4:  440.0 Hz
kare 5:  440.0 Hz
kare 6:  440.0 Hz

```

Aritmetiği elle takip edelim — nedensel seçicinin neden kaybettiği, Viterbi'nin neden kazandığı tek bir karşılaştırmada:

Terim	Değer
3. karede 220 Hz'i seçmenin emisyon kazancı (0,72 – 0,52)	+0,20
440 → 220 geçişi (1200 sent: 0,18 + 0,12)	–0,30
220 → 440 geri dönüşü	–0,30
Net	–0,40

Yalıtılmış bir sıçrama, girişini ve çıkışını birlikte ödemek zorundadır. Kazancı 0,20, maliyeti 0,60 — yol asla oraya sarmaz. **Sürdürülen** bir register değişimi ise girişi bir kez öder, sonraki her kare kâr eder; o yüzden geçer.

Neden "sabit gecikmeli"?

Tam Viterbi tüm kaydı görmeyi gerektirir; canlı çalarken bu imkânsızdır. V2 uzlaşmayı şurada bulur: yalnız **5 kare** ileri bakar. 512 örneklik adımda 48 kHz'de bu ≈ 53 ms sabit gecikme — algılanabilir sınırın hemen altında.

Bir ayrıntı daha: **sessizlik yolu keser**. Dinamik program yalnız en öndeki kareden başlayan kesintisiz seslendirilmiş bloğu kullanır; araya giren bir sessizlik müzikal sürekliliği kopardığı için ileri-bakış orada durur.

```
std::size_t voiced_count = 0;
while (voiced_count < frames_.size() && !frames_[voiced_count].candidates.empty()) {
    ++voiced_count;
}
```

core/src/fixed_lag_tracker.cpp:77–80

3.6 Viterbi'nin görmediği yer — ve neden histerezis hâlâ gerekli

Tasarımın açık kenarı

`resolve_front`, aramayı en öndeki karenin emisyonlarıyla **başlatır**; daha önce yayınlanmış kareye bir geçiş cezası bağlamaz. Yani en öndeki kare için yol, sıçramanın yalnız **çıkış** maliyetini öder, giriş maliyetini değil.

Yukarıdaki tabloda net $-0,40$ idi; giriş maliyeti sayılmasaydı net $+0,20 - 0,30 = -0,10$ olurdu — hâlâ reddedilir, ama marj yarıya iner. Kazancı $0,30$ 'u aşan bir blip bu noktada geçebilirdi.

core/src/fixed_lag_tracker.cpp:83–90

İşte `harmonic_filter` bu yüzden var: Viterbi'den sonra, yayın aşamasında, diğer üç motordakiyle aynı desende bir aşağı-harmonik onay gecikmesi daha uygulanır. İki savunma üst üste durur çünkü ikisi farklı şeyi görür — biri yolun şeklini, diğeri yayınlanan kontura olan mesafeyi.

3.7 Yayın kapıları

Viterbi bir aday seçtikten sonra bile yayın garanti değildir. Sırayla:

Kapı	Kural
RMS kapısı	Kare sessizse yayın yok.
Bırakiş (release)	Kare bırakış anında yakalandıysa yayın yok. Karar 5 kare geciktiği için, kararın ait olduğu karenin bırakış durumu ayrı kuyruklarda saklanır — geç çözüm asla başka bir pencerenin kapısını uygulamaz.
Yüksek register	900 Hz üstü için 3.4'teki ek kanıt barajı.
Güven barajı	$0,70$ — <i>ama</i> son yayınlanan perdenin 90 senti içindeki bir kare $0,55$ 'e düşebilir ("stable weak contour"): sürekliliğin kendisi de kanıttır.
Harmonik histerezis	3.6'daki <code>harmonic_filter</code> .

```
const bool stable_weak_contour = last_published_ && tentative->confidence >= 0.55 &&
    cents(tentative->frequency_hz, last_published->frequency_hz) <= 90;
if (tentative->confidence >= 0.70 || stable_weak_contour) {
    publication = harmonic_filter(*tentative);
}
```

core/src/pitch_engine_v2_session.cpp:490–495

Başlangıç köprüsü (bridge_startup)

Zarif bir ayrıntı: bir notanın atağı çoğu zaman biraz belirsiz başlar, periyodiklik ancak birkaç kare sonra oturur. Bu kareler atılırsa notanın başlangıç zamanı geç görünür — ve öğrenci "geç başladım" sanır.

V2 bunun yerine ilk gerçek yayına kadar olan zayıf kareleri ($\geq 0,55$ güven, ≤ 5 kare, ≤ 90 sent sapma) **tamponlar**; gerçek yayın gelip aynı perdeye yakın çıkarsa, tamponlanan koşu **geriye dönük** serbest bırakılır. Doğrulama gelmezse tampon çöpe gider.

core/src/pitch_engine_v2_session.cpp:579–612

3.8 Bir karenin tam yolculuğu

1536 örnek

```
↓ RMS kapısı + bırakış tespiti (kare başına kuyruklanır) v2_session.cpp:423-460
↓ YIN adayları (periyodiklik  $\geq 0,50$ ) v2_session.cpp:62
↓ + ACF adayı (tek) v2_session.cpp:146
↓ + MPM/NSDF adayları ( $\leq 12$ ) v2_session.cpp:215
↓ + spektral oktav adayı (2x enerji 8x) v2_session.cpp:344
↓ SWIPE' desteği (tek toplu FFT) swipe_prime.cpp:167
↓ — seçim, 1. yarı —
↓ emisyon =  $0,40 \cdot \text{per} + 0,20 \cdot \text{uzlaşma} + 0,40 \cdot \text{SWIPE}'$  v2_session.cpp:379
↓ spektral aday için taban  $0,90$  v2_session.cpp:387
↓ — seçim, 2. yarı: 5 kare tampon —
↓ Viterbi: emisyon + geçiş cezası, en iyi yol fixed_lag_tracker.cpp:83-125
↓ yayın kapıları ( $\text{RMS} \cdot \text{bırakış} \cdot 900 \text{ Hz} \cdot 0,70/0,55$ ) v2_session.cpp:479-499
↓ harmonic_filter (aşağı harmonik onay gecikmesi) v2_session.cpp:493
↓ başlangıç köprüsü + boşluk köprüleme v2_session.cpp:579, 613
PublishedPitchFrame{ zaman, frekans, güven }
```

V2'nin ödünleşimi

Kazandıği: tek karelik hatalar yola oturmaz; dört bağımsız kanıt birbirini denetler; 900 Hz üstünde disiplinli. Ölçümün başında en az oktav hatası yapan motor (3 hata).

Ödediği: ~53 ms sabit gecikme, ve kare başına belirgin biçimde daha çok hesap (dört tahminci + FFT). Çevrimdışı iyileştirme katmanında V2'ye daha gevşek bir geçiş genişliği verilmesinin sebebi de budur: zaten kendi Viterbi'si var, iki kez sıkılaştırmak gerçek hızlı geçkileri bastırır (Bölüm 7).

4. VPM-like

Tadao Yamaoka'nın Vocal Pitch Monitor için anlattığı yöntemin bağımsız uygulaması. Uygulamanın kendi eşikleri yayınlanmadığı için bu bir kopya değil, "VPM-benzeri" bir yeniden inşadır. Üç aşaması var: **normalize otokorelasyon, katlardan periyot rafinesi, spektrumla harmonik düzeltme**. Seçim burada üç ayrı yerde yapılır — hangi ACF tepesi, hangi harmonik akraba, ve hangi kare yayınlanır — ve üçü de birbirinden bağımsız kurallara tabidir.

core/include/klarivision/core/vpm_like.hpp:10–15 · core/src/vpm_like.cpp

4.1 Normalize otokorelasyon (ACF)

Ham ACF ses seviyesine bağlıdır (Bölüm 1.5a). VPM-like bunu, karşılaştırılan iki alt pencerenin enerjilerinin **geometrik ortalamasına** bölerek çözer:

$$r(\tau) = \sum_j x[j] \cdot x[j+\tau] / \sqrt{E_{\text{ön}} \cdot E_{\text{gecikmeli}}}$$
$$E_{\text{ön}} = \sum_{j=0}^{N-\tau} x[j]^2, \quad E_{\text{gecikmeli}} = \sum_{j=\tau}^N x[j]^2$$

sonuç $\approx 1,0 \rightarrow$ dalga biçimi τ örnek sonra kendini tekrarlıyor

İki enerji terimi de önceden hesaplanan bir **ön-ek toplam tablosundan** $O(1)$ sürede okunur — YIN'in kullandığı hilenin aynısı.

```
const auto leading = energy[count]; // [0, count) alt penceresinin enerjisi
const auto delayed = energy[centered.size()] - energy[lag]; // [lag, size) alt penceresinin enerjisi
const auto normalization = std::sqrt(std::max(1e-18, leading * delayed));
double dot = 0.0;
for (int index = 0; index < count; ++index) dot += centered[index] * centered[index + lag];
correlation[lag] = dot / normalization;
```

core/src/vpm_like.cpp:234–241

4.2 Seçim, birinci karar: hangi ACF tepesi?

Burada VPM-like'ın karakteristik hamlesi geliyor ve tek başına motorun kimliğini belirliyor.

En yüksek tepeyi almak neden yanlış

Gerçek periyot τ ise, ACF'de 2τ 'da da bir tepe vardır — ve bu tepe çoğu zaman **daha yüksektir**. Sebep: daha uzun bir gecikmede karşılaştırılan iki pencere daha fazla ortak yapı paylaşır ve normalizasyon terimi de küçülür. Yani "en güçlü tepe" kuralı, sistematik olarak bir oktav aşağıya yanlıdır.

VPM-like'ın çözümü: en güçlü tepeyi bul, ama onu **seçme** — sadece bir **eşik belirlemek** için kullan. Sonra o eşiği geçen **en kısa lag**'i (en yüksek frekansı) seç.

$$\text{ Kabul_eşiği} = \max(0,38, r(\tau_{\text{en güçlü}}) \cdot 0,90)$$

bu eşiği geçen en kısa lag seçilir

PYTHON — KABUL KURALI

```
# --- 1. kabul kurali: en guclunun %90'ini gecen EN KISA lag -----
def kabul_edilen_lag(r, tepeler, en_az_periyodiklik=0.38, yakinlik=0.90):
    en_guclu = max(tepeler, key=lambda t: r[t])
    if r[en_guclu] < en_az_periyodiklik:
        return None, None
    esik = max(en_az_periyodiklik, r[en_guclu] * yakinlik)
    for tepe in tepeler: # tepeler artan lag sirasinda
        if r[tepe] >= esik:
            return tepe, en_guclu
    return en_guclu, en_guclu
```

Tepeler artan lag sırasında bulunduğu için, döngüde eşiği geçen **ilk** tepe otomatik olarak en kısa lag'dir — döngüdeki **break** tek başına tüm kuralı uygular.

PYTHON — DENEME: EN GÜÇLÜ TEPE LAG 200'DE, AMA...

```

r = [0.0] * 500
for lag, deger in [(100, 0.92), (200, 0.95), (300, 0.88), (400, 0.90)]:
    r[lag] = deger
tepeler = [100, 200, 300, 400]
secili, en_guclu = kabul_edilen_lag(r, tepeler)
print(f"en guclu tepe: lag {en_guclu} (r={r[en_guclu]})")
print(f"secilen tepe : lag {secili} (r={r[secili]}) <- daha kısa lag, %90 esigini geciyor")

```

ÇIKTI

```

en guclu tepe: lag 200 (r=0.95)
secilen tepe : lag 100 (r=0.92) <- daha kısa lag, %90 esigini geciyor

```

Eşik = $\max(0,38 \cdot 0,95 \times 0,90) = 0,855$. Lag 100'ün değeri 0,92 bunu geçiyor, dolayısıyla lag 200'e hiç bakılmadan seçiliyor. Bu tablo tam olarak bir oktav belirsizliğidir: lag 200, lag 100'ün iki katı — yani yarı frekans. Kural doğru olanı alıyor.

```

const auto acceptance = std::max(
    config.minimum_periodicity,
    correlation[strongest] * config.near_strongest_ratio
);
auto selected = strongest;
for (const auto peak : peaks) {
    if (correlation[peak] >= acceptance) { selected = peak; break; }
}

```

core/src/vpm_like.cpp:277–286

%90 katsayısı bir ödünleşimdir. Çok gevşek olsaydı (%50 gibi) zayıf, gürültü kaynaklı kısa-lag tepeleri kazanır ve motor tiz yönde hata yapardı. Çok sıkı olsaydı (%99) kural hiç devreye girmez ve motor "en güçlü tepe"ye, yani oktav-aşağı yanlılığa dönerdi.

4.3 Katlardan periyot rafinesi

ACF'nin lag kuantizasyonu (tam sayı örnek sınırı) en çok tiz seslerde göze batır. Parabolik interpolasyon (Bölüm 1.6) yardım eder ama yeterli değildir. Yamaoka'nın çözümü zarif:

Fikir

ACF'nin 2τ , 3τ , 4τ ... civarında da tepeleri vardır. Bu tepelerin her biri, **katına bölündüğünde**, aynı τ periyodunun *bağımsız bir yeniden ölçümüdür*. Üstelik daha uzun bir aralık üzerinden ölçüldükleri için tek-örnek kuantizasyon hatasına daha az duyarlıdır: lag 400'deki bir örneklik hata, dörde bölününce çeyrek örneklik hataya iner. Hepsinin ağırlıklı ortalamasını almak, tek bir tepedeki interpolasyondan daha keskin sonuç verir.

$$\tau_{\text{refine}} = \sum_{m=1}^6 (\tau_m^* / m) \cdot w_m / \sum_m w_m, \quad w_m = r(\tau_m) \cdot m$$

τ_m^* : $m \cdot \tau$ civarında bulunan tepenin parabolik olarak rafine edilmiş konumu

Ağırlık hem tepenin gücüne hem de katın büyüklüğüne orantılıdır: daha yüksek kat = daha uzun ölçüm aralığı = daha güvenilir.

PYTHON – KAT RAFINESİ

```

# --- 2. katlardan periyot rafinesi -----
def kat_rafinesi(r, secili, tau_max, en_az_periyodiklik=0.38, en_yuksek_kat=6):
    ağırlikli = secili * r[secili]
    ağırlık_toplam = r[secili]
    for kat in range(2, en_yuksek_kat + 1):
        beklenen = secili * kat
        if beklenen + 2 >= tau_max:
            break
        yaricap = max(2, secili // 6)
        aralık = range(max(1, beklenen - yaricap), min(tau_max - 1, beklenen + yaricap) + 1)
        yerel = max(aralık, key=lambda t: r[t])
        if r[yerel] < en_az_periyodiklik * 0.8:
            continue
        ağırlık = r[yerel] * kat
        ağırlikli += (yerel / kat) * ağırlık
        ağırlık_toplam += ağırlık
    return ağırlikli / max(1e-12, ağırlık_toplam)

```

core/src/vom_like.cpp:319–341

selected // 6 ile periyodun kendisine orantılı (kayma payı), zayıf katlar (0,38 × 0,8

Üç savunma dikkat çekiyor: arama yarıçapı `selected // 6` ile periyodun kendisine orantılı (kayma payı), zayıf katlar ($0,38 \times 0,8$ altı) ortalamayı kirletmesin diye atlanıyor, ve aralık dışına taşan katlarda döngü kırılıyor.

4.4 Seçim, ikinci karar: hangi harmonik akraba?

ACF *periyodikliği* bulur ama hangi harmoniğin gerçekten temel olduğunu bilmez. Bu aşama ACF'nin kararını gerçek spektrumla sınar: adayın kendisi ve $1/3\times$, $1/2\times$, $2\times$, $3\times$ akrabaları tek tek ölçülür ve **en pes makul olanı** seçilir.

Neden en pes? Çünkü asıl hata modu "eksik/zayıf temel frekans, güçlü harmonikler"dir — tersi çok daha nadirdir. Klarnette bu neredeyse kural (Bölüm 1.8).

Bir aday "makul" sayılmak için üç testi birden geçmeli:

Test	Kural	Eşik
Yerel spektral tepe	Genliği, her iki yanındaki komşu ölçümlerden açıkça büyük olmalı	$\geq \%103$
Mutlak destek	Genliği mutlak olarak yeterince gür olmalı	$\geq 0,005$
Görelî destek	ACF'nin kendi frekansındaki genliğin makul bir kesri olmalı	$\geq \%8$

Yerel tepe testi ince bir noktadır: gerçek bir spektral hat **dardır**. Yanlarına bakıp "orada da bu kadar enerji var mı?" diye sormak, komşu bir harmonikten sızan enerjiyi gerçek bir hat sanmayı engeller.

PYTHON – SPEKTRAL AKRABA SEÇİMİ

```
# --- 3. spektral akraba düzeltmesi -----
def spektral_duzeltme(acf_hz, spektrum, secili, en_guclu,
                    f_min=120.0, f_max=1650.0,
                    goreli=0.080, mutlak=0.005):
    taban = spektrum(acf_hz)
    akrabalar = sorted([acf_hz/3, acf_hz/2, acf_hz, acf_hz*2, acf_hz*3])
    for f in akrabalar:
        if not (f_min <= f <= f_max):
            continue
        genlik = spektrum(f)
        yan = max(1.0, f * 0.012)
        yerel_tepe = genlik >= spektrum(f - yan) * 1.03 and genlik >= spektrum(f + yan) * 1.03
        if not (yerel_tepe and genlik >= mutlak and
                (taban <= 1e-12 or genlik >= taban * goreli)):
            continue
        if f > acf_hz:
            # kısıt 1: yukari terfi yasak
            continue
        if f < acf_hz and secili == en_guclu:
            # kısıt 2: ACF zaten en gucluyu sectiyse
            continue
        return f
    return acf_hz
```

İki kritik kısıt

Fonksiyondaki son iki `continue`, bu pasın gücünü kasten sınırlar.

Kısıt 1 — yukarı terfi yasak

Parlak bir üst harmonik, güçlü bir ACF temel frekansını $2\times/3\times$ 'e **terfi ettiremez**. Düzeltme yalnız aşağı yönlüdür (eksik temeli onarmak için). Sebebi doğrudan klarnet fiziğidir: 3. harmonik zaten çoğu zaman temelden gürdür, dolayısıyla "daha gür olan temeldir" muhakemesi bu çalgıda sistematik olarak yanlış cevabı verir.

```
const auto does_not_promote = config.allow_spectral_promotion ||
    frequency <= autocorrelation_frequency * (1.0 + 1e-12);
```

Kısıt 2 — ACF zaten en güçlüyü seçtiyse, aşağı düzeltme yok

Aşağı doğru spektral düzeltme, *daha erken ve neredeyse-güçlü* bir ACF tepesini onarmak için vardır (yani 4.2'deki kuralın devreye girdiği durum). Seçilen tepe zaten global en güçlü tepeyse, ACF'nin kendisi belirsiz değildir; onu $f/2$ veya $f/3$ ile ezmek, aleyhte holdout kaydında kısa alt-harmonik adacıkları ürettiyordu.

```
const auto does_not_override_strongest_acf =
    frequency >= autocorrelation_frequency * (1.0 - 1e-12) || selected != strongest;
```

core/src/vpm_like.cpp:398–406

Çıkış güveni, seçilen ACF tepesinin yüksekliğidir ve **0,80**'in altındaysa hiçbir şey yayınlanmaz — dört motor içindeki en yüksek kare-içi güven barajı.

4.5 Seçim, üçüncü karar: yayınlanacak mı? (VPMLikeTracker)

Kare tahmincisi kasten **durumsuzdur** (stateless) — aynı pencere her zaman aynı sonucu verir, bu da tanıyı ve testi kolaylaştırır. Kareler arası hafıza ayrı bir sınıfta durur ve tek bir işi vardır: şüpheli bir **aşağı** sıçramayı geciktirmek. Sıradan perde hareketini asla yumuşatmaz.

"Şüpheli" nasıl tanımlanıyor

İki koşulun birleşimi:

$$|cents(yeni, eski)| \geq 650 \text{ ve } yeni/eski \text{ oranı} \in \{ 1/3, 1/2, 2/3 \} \pm 110 \text{ sent}$$

İkinci koşul neden var

Büyük bir düşüşün kendisi şüpheli değildir — müzikte gerçek büyük atlamalar olur. Şüpheli olan, düşüşün **tam olarak bir harmonik oranına** denk gelmesidir. Bir tahminci hata yaptığında rastgele bir yere düşmez; tam olarak $f/2$, $f/3$ veya $2f/3$ 'e düşer. Oran testi, gerçek müzikal atlamayı harmonik hatadan ayıran şeydir.

$2/3$ oranı listede çünkü $3f \rightarrow 2f$ geçişi (onikiliden oktava) klarnette gerçekten olur.

PYTHON — SIÇRAMA TANIMI VE DURUM MAKİNESİ


```
# --- 4. asagi harmonik sicramasi muhafizi -----
def harmonik_sicramasi(onceki, simdiki):
    if simdiki >= onceki or abs(sent(simdiki, onceki)) < 650.0:
        return False
    return any(abs(sent(simdiki / onceki, oran)) <= 110.0
               for oran in (1/3, 1/2, 2/3))

class VPMizleyici:
    def __init__(self, gereken_onay=3):
        self.gereken_onay = gereken_onay
        self.sifirla()

    def sifirla(self):
        self.yayin, self.bekleyen, self.onay = None, None, 0

    def islet(self, tahmin, ust_hat_orani=None):
        if tahmin is None:
            self.sifirla(); return None, "sessizlik"
        if self.yayin is None:
            self.yayin = tahmin; return tahmin, "ilk"
        if not harmonik_sicramasi(self.yayin, tahmin):
            self.bekleyen, self.onay = None, 0
            self.yayin = tahmin; return tahmin, "olagan"
        if ust_hat_orani is not None and ust_hat_orani >= 2.5:
            self.bekleyen, self.onay = None, 0
            return self.yayin, "veto (ust hat 2,5x gur)"
        if ust_hat_orani is None:
            self.bekleyen, self.onay = None, 0
            self.yayin = tahmin; return tahmin, "rakip kanit yok, kabul"
        if self.bekleyen and abs(sent(tahmin, self.bekleyen)) <= 180.0:
            self.onay += 1
        else:
            self.bekleyen, self.onay = tahmin, 1
        if self.onay >= self.gereken_onay:
            self.yayin, self.bekleyen, self.onay = tahmin, None, 0
            return tahmin, "onaylandi"
        return self.yayin, f"beklet ({self.onay}/{self.gereken_onay})"
```

core/src/vpm_like.cpp:23-33 ve 120-163

Üç sonuçlu karar

Durum	Sonuç
Eski (üst) hat hâlâ en az 2,5x daha gur	Doğrudan veto. Eski perde yayınlanmaya devam eder, güven 0,55'e kırılır.
Rakip üst hat kanıtı hiç yok	Yeni pes tahmin anında kabul edilir — veto edilecek bir şey yok.
Üst hat hâlâ gerçek bir spektral tepe	3 ardışık kare onay beklenir (her biri 180 sent içinde); onaya kadar eski perde tekrarlanır.

Ortakdaki satır kritiktir ve motoru "yapışkan" olmaktan kurtarır: gerçek bir pes atak, eski üst hattın spektral enerjisini de beraberinde götürür. O hat kaybolduğu anda veto dayanağı da kalmaz ve yeni nota anında kabul edilir — **geleceğe bakmaya gerek kalmadan**. Kod yorumu bunu özellikle vurguluyor: bu bir veto'dur, spektral terfi değil.

PYTHON – DENEME

```
iz = VPMizleyici()
senaryo = [(440.0, None), (220.0, 3.0), (220.0, 3.0), (220.0, 1.2), (220.0, 1.2), (220.0, 1.2)]
for i, (tahmin, oran) in enumerate(senaryo, 1):
    cikti, sebep = iz.islet(tahmin, oran)
    print(f" kare {i}: tahmin {tahmin:6.1f} -> yayin {cikti:6.1f} [{sebep}]")
```

ÇIKTI

```
kare 1: tahmin 440.0 -> yayin 440.0 [ilk]
kare 2: tahmin 220.0 -> yayin 440.0 [veto (ust hat 2,5x gur)]
kare 3: tahmin 220.0 -> yayin 440.0 [veto (ust hat 2,5x gur)]
kare 4: tahmin 220.0 -> yayin 440.0 [beklet (1/3)]
kare 5: tahmin 220.0 -> yayin 440.0 [beklet (2/3)]
kare 6: tahmin 220.0 -> yayin 220.0 [onaylandi]
```

Senaryoyu okuyalım: 2. ve 3. karelerde 440 Hz hattı hâlâ 3x gur, dolayısıyla 220 Hz doğrudan veto ediliyor ve **onay sayacı bile**

başlamıyor. 4. karede üst hat zayıflıyor (1,2x) — artık veto yok, ama kanıt da tam yok, o yüzden onay süreci başlıyor. Üç onaydan sonra 220 Hz kabul ediliyor. Toplam gecikme 5 kare (≈ 58 ms), ve bu gecikmenin tamamı üst hattın gerçekten kaybolmasını beklemekle geçiyor.

4.6 Oturum katmanı: bırakış, boşluk, atak bekleme

Üretimde VPM-like, `analysis_engine.cpp` içinde kendine ait, diğer motorlardan daha ayrıntılı bir durum makinesiyle sarılır. Ana durumlar:

Durum	Anlamı ve çıkış koşulu
<code>waiting_for_attack</code>	Bir bırakış veya süresi dolmuş boşluktan sonra: hem güvenli bir tahmin hem zarfın son tepesinin %40'ına tırmanması gerekir. İkisi birden olmadan çıkılmaz — gürültülü bir sıçrama tek başına yeni nota sayılmaz.
<code>release_suspected</code>	Yalnız iki sonuç mümkündür: zarf <i>aynı</i> kontura döner (yanlış alarm → tamponlanan boşluk serbest bırakılır), veya <i>farklı</i> bir kontura döner (gerçek yeni atak → eski kontur temizlenir, köprü kurulmaz).
<code>weak_contour_hold</code>	Güvenli tahmin yok ama zayıf bir tahmin izlenen kontura katılıyor: gürültülü zayıf tahmini takip etmek yerine konturun kendi frekansı yeniden yayınlanır. Yani belirsizlik, çizgiyi titretmek yerine dondurur.

core/src/analysis_engine.cpp:600–920

Bu ayrıntı seviyesi tesadüf değil: VPM-like, dinleyici hakemliğinde **sessiz kalması gerektiğinde sessiz kalan** motor olarak ölçüldü. Çevrimdışı katmandaki "başıboş nokta" temizleyicisi bile eşliğini (0,80) doğrudan VPM-like'in kendi yayın barajından alır (Bölüm 7.3).

Bunun bir bedeli var ve ölçümde görünüyor: VPM-like en çok etiketli kusura sahip motor (46). Ama kusurlarının çoğu **boşluktur** — sessiz kalması, yanlış söylemesi değil. Hangisinin daha kötü olduğu bir ürün kararıdır, algoritma kararı değil.

4.7 Bir karenin tam yolculuğu

```
1536 örnek (double'a çevrilir)
↓ DC çıkar + RMS kapaşı                                vpm_like.cpp:184–201
↓ normalize ACF, her  $\tau \in [\tau_{min}, \tau_{max}]$           vpm_like.cpp:234
↓ yerel maksimumlar → tepe listesi                       vpm_like.cpp:246
↓ — seçim 1: en güçlüye %90 yakın EN KISA lag —         vpm_like.cpp:277
↓  $2\tau..6\tau$  tepelerinden ağırlıklı periyot rafinesi      vpm_like.cpp:319
↓ — seçim 2: spektral akrabalar 1/3, 1/2, 1, 2, 3 × —   vpm_like.cpp:361
↓ en pes "makul" olan (3 test + 2 kısıt)                  vpm_like.cpp:387
↓ güven = ACF tepe yüksekliği, < 0,80 ise yayın yok     vpm_like.cpp:459
↓ — seçim 3: VPMLikeTracker veto / onay —               vpm_like.cpp:96
↓ oturum durum makinesi (atak / bırakış / boşluk)        analysis_engine.cpp:600
EngineFrame{ zaman, frekans, güven }
```

5. HAPT v1

"Harmonik-Adaptif Faz-kilitli İzleyici". Diğer üçünün eşiti, dördüncü bağımsız motor. İki fikri kendine özgüdür: **çözünürlük-farkında harmonikler-arası veto** ve **faz kilitli anlık frekans rafinesi**. Birincisi bir seçim kuralıdır ve klarnetin fiziğini doğrudan matematiğe çevirir; ikincisi kareler arası hiçbir hafıza kullanmadan alt-sent doğruluk üretir.

core/include/klarivision/core/hapt.hpp:10–31 · core/src/hapt.cpp · docs/HAPTPitchEngine.md

5.1 Temel: NSDF

HAPT, McLeod'un NSDF'sini kullanır (Bölüm 1.5c) — VPM-like'ın ACF'si gibi normalize, ama iki uçlu:

$$n(\tau) = 2 \cdot \sum_j x[j] \cdot x[j+\tau] / \sum_j (x[j]^2 + x[j+\tau]^2)$$

```
table.values[lag] = 2.0 * correlation / normalisation; // McLeod's NSDF normalisation
```

core/src/hapt.cpp:110

Küçük ama düşünülmüş bir ayrıntı: tablonun iki ucuna birer sıfır dolgu konur, böylece **tam $\tau_{\min}$** 'deki bir tepe de tepe sayılabilir. Bunsuz, $f_{\max}$ 'a çok yakın gerçek bir temel frekans hiç aday olamazdı.

core/src/hapt.cpp:92–98

0,05'in üstündeki her yerel maksimum bir adaydır. Şimdi asıl iş: bu adaylar arasından hangisinin *gerçek* temel frekans olduğunu seçmek.

5.2 Seçim: harmonikler-arası veto

NSDF, gerçek periyodu katlarından ayıramaz — 2τ , 3τ , $k\tau$ hepsi neredeyse aynı derecede periyodik görünür (Bölüm 1.5 kutusu). Diğer motorlar bunu enerji kıyaslamalarıyla çözmeye çalışır. HAPT bambaşka bir soru sorar:

Sorulan soru

"Bu aday gerçekten bir oktav (veya bir onikili) **fazla pes** olsaydı, spektrumda **başka nerede** enerji olurdu? Şimdi oraya bakalım — var mı?"

Aday frekansı f olsun. f gerçek temelse, enerji f , $2f$, $3f$... noktalarındadır — **ve başka hiçbir yerde değildir**. Ama gerçek temel $2f$ ise (yani f bir oktav fazla pesse), gerçek notanın kısımleri $2f$, $4f$, $6f$ 'tedir; f 'in "yarım ızgara" noktaları — $0,5f$, $1,5f$, $2,5f$ — o zaman **boşluğa** denk gelir. Tersine, f gerçekten temelse o noktalarda da (kendi tek harmonikleri sayesinde) enerji bulunur.

$$\text{yarım_veto} = E_{\text{yarım ızgara}} / (E_{\text{yarım ızgara}} + E_{\text{kendi harmonikleri}})$$

üçte_bir_veto: aynısı $(k-\frac{1}{3})\cdot f$ ve $(k-\frac{2}{3})\cdot f$ problemleri için

0 = veto yok; 1'e yakın = "bu aday muhtemelen bir kat fazla pes"

PYTHON – IZGARA VETOSU VE TEK HARMONİK DOLULUĞU

```
# --- 1. bir adayın veto kanıtları -----
def ızgara_vetosu(spektrum, f, kaydirmalar, harmonik_sayisi=5):
    """kaydirmalar: yarım ızgara için [-0.5], üçte bir için [-1/3, -2/3]"""
    kendi = sum(spektrum(k * f) ** 2 for k in range(1, harmonik_sayisi + 1))
    ızgara = sum(spektrum((k + kay) * f) ** 2
                 for k in range(1, harmonik_sayisi + 1) for kay in kaydirmalar)
    toplam = kendi + ızgara
    return (kirp01(ızgara / toplam) if toplam > 1e-18 else 0.0), kendi

def tek_harmonik_dolulugu(spektrum, f, harmonik_sayisi=5, esik=0.10):
    genlikler = {k: spektrum(k * f) for k in range(1, harmonik_sayisi + 1)}
    en_gur = max(genlikler.values())
    tekler = [k for k in (1, 3, 5) if k in genlikler]
    if not tekler:
        return 1.0
    mevcut = sum(1 for k in tekler if en_gur > 1e-12 and genlikler[k] >= esik * en_gur)
    return mevcut / len(tekler)
```

```
for (int k = 1; k <= harmonic_count; ++k) {
    harmonic_multiples.push_back(k);           // adayin kendi k. harmonigi
    half_multiples.push_back(k - 0.5);         // "yarim ızgara" probu
    third_multiples.push_back(k - 1.0 / 3.0);   // "üçte bir ızgara" problemleri
    third_multiples.push_back(k - 2.0 / 3.0);
}
```

core/src/hapt.cpp:241–246

Çözünürlük farkındalığı

Burada HAPT'ı özel kılan ikinci katman var. Yarım-ızgara probu, ancak pencere o frekans aralığını **çözebiliyorsa** anlamlıdır. Pencere kısaysa, prob komşu harmoniğin sızıntısını ölçer ve saçmalar — üstelik kendinden emin biçimde saçmalar.

HAPT bunu Hann ana lobu genişliğinden (Bölüm 1.7) hesaplar ve **ölçüm yöntemini değiştirir**:

$$\text{yarım ızgara tabanı} = 3 \cdot (2f_s/N) , \quad \text{üçte bir ızgara tabanı} = 4,5 \cdot (2f_s/N)$$

```
const auto half_width = hann_main_lobe_half_width_hz(samples.size(), sample_rate);
const auto half_grid_floor_hz = 3.0 * half_width;
const auto third_grid_floor_hz = 4.5 * half_width;
```

core/src/hapt.cpp:185–187

Aday bu tabanın **üstündeyse** doğrudan spektral prob kullanılır. **Altındaysa** zaman düzlemine düşülür ve NSDF'nin katındaki değeri kıyaslanır:

$$\text{yarım_veto} = \text{clip}(n(2\tau) / n(\tau) - 1, 0, 1)$$

oktav-aşağı tepe kendi tepemizden ne kadar güçlüyse o kadar şüphe

```
const auto lower = nsdf_at_lag(*table, static_cast<int>(std::lround(2.0 * refined_lag)));
if (lower >= 0.0 && periodicity > 1e-9) {
    half_veto = clamp01(lower / periodicity - 1.0);
}
```

core/src/hapt.cpp:262–265

Bu "elimdeki pencereyle hangi kanıt ölçebilirim, ölçemiyorsam hangi ikinci en iyi kanıtı düşerim?" refleksi, diğer üç motorda yok. Onlar tek bir kanıt türüne bağlıdır ve pencere yetersiz kaldığında sessizce yanlış ölçerler.

Tek harmonik doluluğu — ve çift harmoniklerin serbest bırakılması

Klarnet kapalı silindirdir: tek kısımlar (1., 3., 5.) tonu taşır, çiftler fizik gereği zayıftır. HAPT bunu doğrudan bir ölçüye çevirir ve **yalnız {1, 3, 5}'e bakar** — çiftlere hiçbir kısıt koymaz.

$$\text{doluluk} = (\text{mevcut olan tek harmonik sayısı}) / (\text{ölçülebilen tek harmonik sayısı})$$

"mevcut" = genliği, en gür harmoniğin en az %10'u

Neden çiftlere bakılmıyor?

Çünkü tam da o, iki durumu birbirine karıştıran testtir. "Çift harmonikleri zayıf" olmak, klarnette hem **gerçek bir temel frekansın** hem de **periyodu ikiye katlanmış bir hayaletin** imzasıdır — ikisi de bu testi aynı şekilde geçer. Ayırt eden şey çiftlerin zayıflığı değil, **teklerin varlığıdır**. Çiftlere kısıt koymak, doğru cevabı yanlış cevapla aynı kefeye koymak olurdu.

Anlamlılık ağırlığı: neden iki geçiş gerekiyor

İnce bir tuzak var ve tek geçişli bir puanlama buna düşer.

Gerçek hiçbir spektral içeriğe denk gelmeyen bir aday düşünün — gerçek tondan çok pes bir alt-harmonik. Onun 1., 3., 5. harmonikleri de **hiçbir yere** denk gelmez; hepsi sadece pencere sızıntısı gürültüsüdür. Ama gürültü kendi içinde *tutarlıdır*: hepsi birbirine yakın büyüklüktedir. Dolayısıyla "{1,3,5} en gür harmoniğin %10'unu geçiyor mu?" testi **%100 dolu** cevabını verir. Sahte bir mükemmel skor.

HAPT bunu iki geçişle çözer. Birinci geçişte her adayın **mutlak** harmonik enerjisi de ölçülür; ikinci geçişte doluluk terimi, adayın karedeki en güçlü harmonik enerjisinden ne kadar pay aldığına göre ağırlıklandırılır:

$$\text{anamlılık} = \text{clip}(E_{\text{aday}} / E_{\text{en güçlü aday}}, 0, 1)$$

$$\text{doluluk_carpanı} = 0,35 + 0,65 \cdot \text{doluluk}$$

$$\text{etkin_carpanı} = 1 - \text{anamlılık} \cdot (1 - \text{doluluk_carpanı})$$

$\text{anamlılık} \approx 0 \rightarrow \text{carpan } 1\text{'e (nötr) gider}$ · $\text{anamlılık} \approx 1 \rightarrow \text{tam klarnet biçimli modülasyon}$

Son satırdaki ters çevirme kritiktir: **anlamsız bir adaya doluluk ödülü de ceza da verilmez**, carpanı 1'e (nötr) çekilir. Onu eleyen şey doluluk değil, `anamlılık_carpanı` 'nın kendisidir ($0,10 + 0,90 \cdot \text{anamlılık}$).

Nihai puan

$$\text{puan} = \text{clip}[\text{periyodiklik} \cdot \text{etkin_doluluk_carpanı} \cdot (0,10 + 0,90 \cdot \text{anamlılık}) \cdot (1 - 0,9 \cdot \text{yarım_veto}) \cdot (1 - 0,9 \cdot \text{üçte_bir_veto}) + \text{süreklilik_bonusu}]$$

süreklilik bonusu: 700 sente kadar doğrusal azalarak +0,10'a kadar

Yapı **çarpımsaldır**, toplamsal değil — ve bu bilinçli bir tercih. Toplamsal bir puanda güçlü bir terim zayıf bir terimi telafi edebilir; çarpımsal bir puanda **her koşul ayrı ayrı sağlanmak zorundadır**. Bir vetoyu yemiş aday, ne kadar periyodik olursa olsun toparlanamaz. Yine de her veto en fazla %90 kesebilir (0,9 katsayısı): hiçbirisi mutlak ret değildir.

PYTHON – İKİ GEÇİŞLİ PUANLAMA

```
# --- 2. iki gecisli puanlama -----
def hapt_puanla(adaylar, spektrum, onceki_hz=None):
    on_hazirlik = []
    en_yuksek_enerji = 0.0
    for a in adaylar:
        f = a["frekans"]
        yarim_veto, _ = izgara_vetosu(spektrum, f, [-0.5])
        ucte_bir_veto, _ = izgara_vetosu(spektrum, f, [-1/3, -2/3])
        doluluk = tek_harmonik_dolulugu(spektrum, f)
        en_yuksek_enerji = max(en_yuksek_enerji, kendi_enerjisi)
        on_hazirlik.append({**a, "yarim_veto": yarim_veto, "ucte_bir_veto": ucte_bir_veto,
                              "doluluk": doluluk, "enerji": kendi_enerjisi})

    sonuc = []
    for a in on_hazirlik:
        anamlilik = kirp01(a["enerji"] / en_yuksek_enerji) if en_yuksek_enerji > 1e-18 else 0.0
        anamlilik_carpani = 0.10 + 0.90 * anamlilik
        doluluk_carpani = 0.35 + 0.65 * a["doluluk"]
        etkin_doluluk = 1.0 - anamlilik * (1.0 - doluluk_carpani)
        bonus = 0.0
        if onceki_hz:
            bonus = max(0.0, min(0.10, 0.10 * (1.0 - sent(a["frekans"], onceki_hz) / 700.0)))
        puan = kirp01(a["periyodiklik"] * etkin_doluluk * anamlilik_carpani
                      * (1.0 - 0.9 * a["yarim_veto"]) * (1.0 - 0.9 * a["ucte_bir_veto"])) + bonus)
        sonuc.append({**a, "anamlilik": anamlilik, "puan": puan})
    return sorted(sonuc, key=lambda a: -a["puan"])
```

core/src/hapt.cpp:367–370

Deneyelim: yüksek periyodiklik her zaman kazanmaz

Gerçek nota 440 Hz, klarnet: tek harmonikler gür (440, 1320, 2200), çiftler zayıf. Rakip aday 220 Hz — klasik bir oktav-aşağı hayalet, ve **periyodikliği daha yüksek** (0,93 vs 0,91).

PYTHON – DENEME

```

hatlar = {440: 0.50, 880: 0.05, 1320: 1.00, 1760: 0.04, 2200: 0.45}
def spektrum(f):
    for hat, genlik in hatlar.items():
        if abs(f - hat) < 12:
            return genlik
    return 0.002 # gurultu tabani

adaylar = [{"frekans": 220.0, "periyodiklik": 0.93},
            {"frekans": 440.0, "periyodiklik": 0.91}]
for a in hapt_puanla(adaylar, spektrum):
    print(f"{a['frekans']:.1f} Hz | periyodiklik {a['periyodiklik']:.2f} "
          f"| yarim veto {a['yarim_veto']:.2f} | doluluk {a['doluluk']:.2f} "
          f"| anlamlilik {a['anlamlilik']:.2f} | PUAN {a['puan']:.3f}")

```

ÇIKTI

```

440.0 Hz | periyodiklik 0.91 | yarim veto 0.00 | doluluk 1.00 | anlamlilik 1.00 | PUAN 0.910
220.0 Hz | periyodiklik 0.93 | yarim veto 0.00 | doluluk 0.00 | anlamlilik 0.17 | PUAN 0.211

```

Sonuç dört kattan fazla fark: 0,910 vs 0,211. Nasıl olduğunu terim terim izleyelim:

Terim	440 Hz	220 Hz	Neden
periyodiklik	0,91	0,93	220 Hz burada <i>önde</i> — tek başına bakılsa kazanırdı
tek doluluk	1,00	0,00	220'nin teklere (220, 660, 1100) gürültü tabanında; 440'ınkiler (440, 1320, 2200) gür
anlamlılık	1,00	0,17	220'nin harmonik ailesi karedeki enerjinin ancak %17'sini açıklıyor
puan	0,910	0,211	çarpımsal yapı: iki zayıf terim birbirini büyütür çarpıyor

Dikkat: **yarım veto burada devreye bile girmedir** (ikisi de 0,00). Kararı veren tek harmonik doluluğu ile anlamlılık oldu. Bu, üst üste konmuş savunmaların işleyişini iyi gösteriyor: farklı kanıtlar farklı durumlarda tetiklenir, hepsinin aynı anda çalışması gerekmez.

5.3 Faz kilitli anlık frekans

Parabolik interpolasyon (Bölüm 1.6) periyodu alt-örnek doğrulukla verir, ama bu hâlâ bir *tahmindir*. HAPT son bir rafine daha yapar ve bunun için **fazı** kullanır — bu bilgiyi kullanan tek motor odur.

Fikir

Aynı 1536 örneklik pencerenin içinden, 512 örnek kaydırılmış **iki alt pencere** (her biri 1024 örnek) alın. Aynı frekansta prob atın. Prob frekansı gerçek frekansa tam eşitse, iki ölçümün fazı da aynı olur. Fark varsa, faz kayması doğrudan frekans sapmasıyla orantılıdır — çünkü frekans, fazın zamana göre türevidir.

$$\Delta\phi = \arg(X_{\text{geç}} \cdot \text{conj}(X_{\text{erken}}))$$

$$df = -\Delta\phi \cdot f_s / (2\pi \cdot 512)$$

±93,75 Hz belirsizliksiz aralık (docs/HAPTPitchEngine.md:73)

```

const auto phase_difference = std::arg(chosen_late * std::conj(chosen_early));
const auto deviation_hz = -phase_difference * sample_rate /
    (2.0 * std::numbers::pi * static_cast<double>(kIFShiftSamples));
const auto refined_probe_frequency = chosen_harmonic * frequency_hz + deviation_hz;
const auto refined_frequency = refined_probe_frequency / chosen_harmonic;

```

core/src/hapt.cpp:460–466

Prob, temel frekansta değil, **1., 3. veya 5. harmonikten en güçlüsünde** atılır — orası fazın en temiz okunduğu yerdir. Sonuç katına bölünerek temel frekansa geri indirilir. Harmonikte ölçmenin ek bir faydası var: sapma da k katı büyür, yani aynı faz çözünürlüğüyle k kat daha hassas ölçüm yaparsınız.

Belirsizliksiz aralık nereden geliyor? Faz yalnız $(-\pi, \pi]$ içinde tek anlamlı okunur. $\Delta\phi = \pm\pi$ 'ye karşılık gelen sapma $\pm f_s/1024$ 'tür, yani pencerenin toplam genişliği $f_s/512$ olur — 48 kHz'de **93,75 Hz** ($\pm 46,9$ Hz). Tasarım notu bu genişliği " $\pm 93,75$ Hz" diye anıyor (docs/HAPTPitchEngine.md:73). Bunun ötesinde faz sarar (wrap) ve ölçüm sessizce yanlış döner — sarmayı tespit etmenin bir

yolu olmadığı için aşağıdaki üçüncü kilit, büyük bir düzeltmeyi düzeltmek yerine **reddeder**.

Üç güvenlik kilidi

Kilit	Gerekçe
Periyodiklik $\geq 0,75$	Fazı okumaya geçecek kadar temiz bir ton olmalı; gürültüde faz anlamsızdır.
İki alt pencere arası genlik farkı ≤ 6 dB	Atak/geçiş karesinde enerji hızla değişir; faz kayması o zaman frekansı değil zarfı ölçer.
Düzeltilme ≤ 40 sent	Daha büyük bir "düzeltme" muhtemelen faz sarması belirsizliğidir, gerçek rafine değil. Kilit, sarmayı tespit edemeyeceğimiz için sonucu reddetmeyi seçer.

core/src/hapt.cpp:448–470 · eşikler: hapt.hpp:42–44

Ayrıca rafine yalnız **tam ve kurtarılmamış** bir pencerede denenir; kurtarma alt penceresinden gelen bir tahmin için hiç çalıştırılmaz — kısa pencerede faz ölçümü zaten güvenilir değildir.

5.4 Atak ve sönüm kurtarması

Tam pencere yeterince güvenli bir şey bulamadıysa, sebep pencerenin *içinde* notanın başlıyor veya bitiyor olması olabilir — periyodiklik tüm pencereye yayıldığı için sulanmıştır. Bir kareyi tamamen kaybetmek yerine HAPT iki kurtarma denemesi yapar:

1. **Atak varsayımı:** yalnız pencerenin **son** 768 örneğini yeniden analiz et (sinyal yeni periyodikleşti).
2. **Sönüm varsayımı:** yalnız pencerenin **ilk** 768 örneğini analiz et (sinyal periyodikti, şimdi ölüyor).

Başarılı bir kurtarma, kanıtın yalnız bir kısmını gördüğümüz için puanı **%15 kırılır** — bulunduğu şey gerçek ama daha az kanıta dayanıyor, ve puan bunu dürüstçe yansıtmalı.

core/src/hapt.cpp:517–562

5.5 İki eşikli histerezis

HAPT'ın "eksik seslendirilmiş kare" sorununa (diğer üç motorun ortak zaafı) cevabı budur: **tek eşik yerine iki eşik**.

izlenen kontura ≤ 180 sent uzaklıkta → eşik 0,32 (*sustain*)
taze veya uzak bir aday → eşik 0,55 (*onset*)

PYTHON – İKİ EŞİKLİ KARAR

```
# --- 3. iki eşikli histerezis -----
def yayınlanabilir(aday, onceki_hz, onset=0.55, sustain=0.32, kontur_sent=180.0):
    yakin = onceki_hz is not None and sent(aday["frekans"], onceki_hz) <= kontur_sent
    esik = sustain if yakin else onset
    return aday["periyodiklik"] >= esik, esik
```

ÇIKTI (AYNI ADAY, PERİYODIKLIK 0,40)

```
kontur yok (onset)          esik 0.55 -> sessiz
konturun 180 senti içinde (sustain) esik 0.32 -> yayın
```

Aynı ölçüm, iki farklı bağlamda iki farklı sonuç veriyor. Bu keyfi değil, doğrudan bir olasılık ifadesi: sürmekte olan bir notanın devam ediyor olması, hiç yoktan yeni bir notanın başlamış olmasından çok daha olasıdır. Süreklilik burada da **kanıt** sayılır — V2'nin "stable weak contour" kuralının (Bölüm 3.7) HAPT'taki karşılığı.

core/include/klarivision/core/hapt.hpp:50–52

5.6 HAPTTracker: bırakış tespiti ve sıçrama muhafızı

Sustain eşliğinin kasten gevşek tutulmasının bir bedeli var: sönen bir reverb kuyruğu da o eşığı geçebilir. Bu yüzden HAPT'ın bırakış tespiti diğerlerinden bir adım daha ileridir ve **iki** tetikleyicisi vardır:

Tetikleyici	Yakaladığı durum
2 ardışık %20 keskin düşüş	Sıradan nota bitışı (release) — hızlı ve sert
RMS, son tepesinin %35'inin altına indi	Yumuşakça sönen oda reverb kuyruğu — hiçbir karede "keskin" düşmez ama toplamda çok alçalır

Kurtulma koşulu: zarfın son tepenin %40'ına geri tırmanması.

```
if (release_falls_ >= 2 || decayed_below_recent_peak) release_active_ = true;
if (release_active_ && rms >= release_rms_peak_ * config_.release_recovery_ratio) {
    release_active_ = false; release_falls_ = 0;
}
```

core/src/hapt.cpp:653–659

Kritik sıra: bırakış tespiti **tahminciden önce ve ondan bağımsız** çalışır. Yani hâlâ sönmekte olan bir kuyruk, tahmincinin "yeterince gür ve periyodik" dediği bir karede bile bastırılır. Bu, gevşek sustain eşiğinin bedelini ödeyen mekanizmadır.

Aşağı harmonik sıçraması muhafızı VPM-like'inkiyle aynı desendir; farkları:

	VPM-like	HAPT
Bakılan oranlar	1/3, 1/2, 2/3	1/3, 1/2
Oran toleransı	110 sent	30 sent
Gereken onay	3 kare	2 kare

HAPT daha dar tolerans ve daha az onayla çalışır çünkü ızgara vetosu zaten kare içinde çok iş yapıyor; muhafız burada ikincil bir emniyet kemeridir, ana savunma değil.

core/src/hapt.cpp:52–66 ve 683–708

5.7 Bir karenin tam yolculuğu

1536 örnek

- ↓ DC çıkar + RMS kapısı hapt.cpp:492
- ↓ NSDF tablosu (kenar dolgulu) hapt.cpp:76
- ↓ yerel maksimumlar $\geq 0,05 \rightarrow$ hipotez listesi hapt.cpp:172
- ↓ — seçim, 1. geçiş —
- ↓ yarım/üçte-bir veto (çözünürlüğe göre spektral/NSDF) hapt.cpp:190
- ↓ tek harmonik doluluğu {1,3,5} + mutlak enerji hapt.cpp:285
- ↓ — seçim, 2. geçiş —
- ↓ anlamlılık ağırlığı + çarpımsal nihai puan hapt.cpp:327
- ↓ atak / sönüm kurtarması (768 örneklik alt pencere) hapt.cpp:517
- ↓ iki eşikli histerezis (0,55 onset / 0,32 sustain) hapt.cpp:564
- ↓ faz kilitli anlık frekans rafinesi (≤ 40 sent) hapt.cpp:404
- ↓ HAPTTracker: bırakış tespiti + sıçrama muhafızı hapt.cpp:637
- ↓ boşluk köprüleme (yin_v1 ile ortak; taban 0,60/0,30) analysis_engine.cpp:274

EngineFrame{ zaman, frekans, güven }

HAPT'ın gevşetilmiş köprü ucu

Ortak [append_bridged](#) çağrısında yalnız HAPT için **asimetrik** bir eşik verilir: güçlü uç 0,60'ı geçmeli ama *zayıf* uç için taban 0,30'a indirilir. Gerekçe kodda yazılı: bir kopmanın çıkış kenarı, tam da motorun kendi güveninin kesinti tarafından en çok bozulduğu yerdir; iki ucun da aynı barajı geçmesini istemek gerçek kısa boşlukları eksik köprüler.

core/src/analysis_engine.cpp:281–288, 933–934

6. pYIN — çevrimdışı referans

*pYIN, uygulamada kullanıcıya sunulan dört motordan biri değildir; **çevrimdışı referans eğrisi** olarak kullanılır. Kendi uygulamamız değil, librosa'nın uygulamasıdır — ama neden bu kadar iyi olduğunu bilmek, dört motorun tasarım kararlarını anlamak için gereklidir.*

src/klarivision/pitch/pyin.py – librosa.pyin adaptörü

6.1 YIN'den farkı: tek eşik yerine olasılık dağılımı

Klasik YIN'de tek bir kritik karar vardır: "CMND eğrisinde 0,15 eşikinin altına düşen ilk dip". Bu eşik **keyfidir** ve tek bir yanlış seçim tüm kareyi mahveder.

pYIN'in birinci fikri: **eşiği seçme, hepsini dene**. Farklı eşik değerleri üzerinde bir olasılık dağılımı (beta dağılımı) tanımlanır ve her eşik değerinin verdiği aday, o eşik olasılığıyla ağırlıklandırılır. Sonuç, tek bir frekans değil, kare başına **ağırlıklı bir aday kümesidir**.

6.2 İkinci ve asıl fikir: tüm kayıt üzerinde Viterbi

Aday kümeleri sonra bir **gizli Markov modeline** (HMM) verilir ve tüm kayıt boyunca en olası yol çözülür.

Neden pYIN oktav hatası yapmıyor

Yalıtılmış bir 1200 sentlik sapma, **küresel** bir yol arayışında geçiş maliyetini asla geri ödeyemez. Bir kare "oktav aşağı" demek için, o kareye girmenin ve o kareden çıkmanın maliyetini birlikte ödemek zorundadır. Nedensel bir motor bunu göremez, çünkü çıkış maliyetini henüz bilmez.

Ölçüm bunu doğruluyor: Şükrü Tunar hakemlik kümesinde işaretlenen 47 oktav hatasının **33'ü yalnız 1–3 kare sürüyor** (medyan 2, en uzun 9) ve doğru izlemenin tam ortasında oturuyor. pYIN bu tür hataları yapmıyor; çünkü kararı tek karede vermiyor.

Bu gözlem, Bölüm 7'deki çevrimdışı iyileştirme katmanının doğrudan gerekçesidir.

6.3 Kullanım biçimi ve maliyeti

```
frequencies, voiced, probability = librosa.pyin(
    samples[analysis_start:analysis_end],
    fmin=settings.minimum_frequency_hz, fmax=settings.maximum_frequency_hz,
    sr=sample_rate, frame_length=settings.frame_length, hop_length=settings.hop_length,
    resolution=settings.resolution_cents / 100, center=False, fill_na=np.nan,
)
```

src/klarivision/pitch/pyin.py:67–77

Ayar	Değer	Not
frame_length	2048	@22,05 kHz = 93 ms
hop_length	110	5 ms — üretim motorlarından ~2 kat sık
frekans aralığı	110–2000 Hz	üretim motorlarından geniş
resolution	5 sent	HMM durum ızgarasının çözünürlüğü

Kayıt 3 saniyelik parçalara bölünerek işlenir; her parça bir çerçeve boyu geriden başlar (**overlap**) ve yalnız çekirdek bölgesi tutulur — böylece parça sınırlarında kenar etkisi oluşmaz.

src/klarivision/pitch/pyin.py:64–86

Neden pYIN canlı modda kullanılmıyor

Çünkü küresel Viterbi **tüm kaydı görmeyi** gerektirir. Bu, canlı çalarken tanıma gereği imkânsızdır. Dört üretim motorunun tamamının nedensel (causal) olmasının sebebi budur; V2'nin 5 kareli sabit gecikmesi, bu duvara verilen en yakın uzlaşmadır.

7. Çevrimdışı iyileştirme katmanı

`PitchEngineProfile::offline_track` uzun süre yer tutucuydu. Artık dolu: Çalışma (Study) modunda, nedensel izin üzerinde **geleceğe bakan** iki temizlik pası çalışıyor. Canlı yol hiç değişmedi — `analyse_causal` ve her `ProductionPitchSession` gecikme sözleşmesini koruyor. Bu bölüm, dört motorun da üstüne oturan ortak bir seçim katmanıdır.

7.1 Neden gerekti

Bölüm 6.2'deki ölçüm: işaretli 47 oktav hatasının **33'ü yalnız 1–3 kare** sürüyor (medyan 2, en uzun 9 kare / 96 ms), hepsi doğru izlemenin ortasında yalıtılmış sıçramalar. Nedensel bir motor bunları göremez, çünkü bir hatanın *ne kadar sürdüğünü* ancak sonrasını görünce anlarsınız. pYIN görebiliyor. Öyleyse çevrimdışı analizde biz de görebiliriz.

Dikkat edilecek nokta: bu bir **kestirici** sorunu değil, **karar verici** sorunudur. Aday üretimi zaten doğru cevabı listede tutuyordu; seçim onu bulamadı çünkü seçim anında yeterli bağlam yoktu.

7.2 Harmonik yol iyileştirmesi

Yayınlanmış her kare için, o karenin kendi analiz penceresi bulunur ve **harmonik ailesi** ölçülür: yayınlanan frekans ve onun $\frac{1}{5}x$, $\frac{1}{4}x$, $\frac{1}{3}x$, $\frac{1}{2}x$, $2x$, $3x$, $4x$, $5x$ akrabaları.

Merdiven neden 5x'e kadar uzanıyor

Başlangıçta aile $\{\frac{1}{5}, \frac{1}{2}, 2, 3\}$ idi. Sorun şu: kapalı silindirin gerçekten ürettiği kısımler daha uzağa gider, ve nedensel pas bir kareyi kendi **4.** veya **5.** alt-periyodunda yayınlatabilir. Böyle bir karenin geri dönüş yolu yoktu — 121,25 Hz yayınlanmış bir kare için eski küme yalnız 242,5 ve 363,75 ürettiyordu; doğru cevap olan 604,5 Hz **erişilebilir bile değildi** ve Viterbi yanlış cevaplar arasından seçiyordu.

Neden nedensel katmanda değil? Ölçülen iki durum spektral olarak neredeyse aynı, doğru cevapları zıt: 116,379 sn'de 121,27 Hz'in $5x$ baskınlık oranı 14,5 (yanlış cevap), 112,293 sn'de doğru cevap olan 264,55 Hz'in $2x$ baskınlık oranı 10,2 (doğru cevap). Salt spektral hiçbir kural ikisini ayıramaz — ayırt edici olan **zamansal bağlamdır**. Bu yüzden daha uzağa uzanmak yalnız burada güvenli: kararı geçiş maliyeti verir, ve bir $5x$ alternatifi ancak komşu kareler zaten oradaysa kendini amorti eder.

docs/TEST_BASELINE.md — "Çevrimdışı harmonik yol merdiveni", 31 Ağustos 2026

Kritik tasarım kararı: spektral kanıt *fiyattır*, kapı değil

Alternatifleri spektral desteğe göre **elemek** denendi ve yanlış. Onarıma gerçekten muhtaç karelerde doğru temel frekans, ailenin en güçlü üyesinin çoğu zaman **%2'sinin altında** ölçülüyor — nedensel motorun o kareyi zaten bu yüzden yanlış bilmesinin sebebi tam olarak budur, ve bilginin yalnızca yolda bulunduğu durum tam olarak odur. Dolayısıyla her harmonik alternatif **erişilebilir kalır**; spektral desteği olmayan sadece *daha pahalıya* gelir.

$$\text{destek} = E_{\text{üye}} / E_{\text{ailenin en güçlüsü}}$$
$$\text{alternatif_puanı} = \text{güven} \cdot 0,72 \cdot [0,30 + 0,70 \cdot \text{clip}(\text{destek}, 0, 1)]$$

nedensel kararın kendisi her zaman 1,00 katsayısıyla, indirimsiz yansır

İki katsayı iki ayrı iş yapıyor:

- **0,30 tabanı:** hiçbir spektral desteği olmayan bir alternatif bile puanının %30'unu korur. Sıfır olsaydı, tam da onarıma muhtaç kareler elenirdi.
- **0,72 iskontosu:** nedensel karar her zaman önde başlar. Bir kare ancak **yol sürekliliği** talep ettiğinde yer değiştirir, yerel kanıt talep ettiğinde değil.

PYTHON — AILE KURULUMU VE FİYATLANDIRMA

```
# --- 1. her kare için harmonik ailesi ve fiyatı -----
def katmanlar(iz, spektrum, f_min=120.0, f_max=1500.0):
    cikti = []
    for indeks, kare in enumerate(iz):
        if not kare["frekans"]:
            continue
        yayin, guven = kare["frekans"], max(kare["guven"], 1e-3)
        secenekler = [{"frekans": yayin, "puan": guven}] # nedensel karar: indirimsiz
        # Kapalı silindirin gerçekten ürettiği merdiven: 4x ve 5x de dahil.
        oranlar = (0.2, 0.25, 1/3, 0.5, 2.0, 3.0, 4.0, 5.0)
        aile = [yayin] + [yayin * o for o in oranlar]
        aile = [f for f in aile if f_min <= f <= f_max]
        enerjiler = {f: spektrum(indeks, f) for f in aile}
        en_guclu = max(enerjiler.values()) if enerjiler else 0.0
        for f in aile[1:]:
            destek = enerjiler[f] / en_guclu if en_guclu > 0 else 0.0
            arka = TABAN + (1.0 - TABAN) * max(0.0, min(1.0, destek))
            secenekler.append({"frekans": f, "puan": guven * ISKONTO * arka})
        cikti.append({"indeks": indeks, "secenekler": secenekler})
    return cikti
```

```
const double backing = kOfflineUnsupportedFloor +
    (1.0 - kOfflineUnsupportedFloor) * std::clamp(support, 0.0, 1.0);
const double scaled = confidence * kOfflineAlternativeDiscount * backing;
options.push_back({family[member].frequency, scaled, std::log(std::max(scaled, 1e-6))});
```

core/src/analysis_engine.cpp:1123–1126 · sabitler: analysis_engine.cpp:992–1001

Geçiş maliyeti neden karesel?

$$\text{geçiş}(a \rightarrow b) = -\min((\text{cents}(a,b) / w)^2, 12)$$

$$w = 150 \text{ sent (nedensel motorlar)} \cdot 250 \text{ sent (V2)}$$

V2'nin canlı Viterbi'si doğrusal ceza kullanıyordu (Bölüm 3.5); burada karesel. Fark bilinçli:

Doğrusal vs karesel

Doğrusal cezada, 1200 sentlik tek bir sıçrama ile 100 sentlik on iki küçük adım **aynı** maliyeti öder. Ama bunlar müzikal olarak taban tabana zıt şeylerdir: biri bir hata, diğeri bir gezinti. Karesel cezada tek büyük sıçrama $(1200/150)^2 = 64$ öderken, on iki küçük adım $12 \times (100/150)^2 \approx 5,3$ öder — on iki kat fark. Karesel form, "gerçek müziğin küçük adımlarla ilerlediği, hataların ise tek hamlede sıçradığı" gözlemini doğrudan matematiğe çevirir.

Üst sınır (12) ise ters yönde koruma: tek bir çılgın kare, tüm yol puanını domine edip etrafındaki doğru kararları da bozmasın.

V2'ye daha gevşek genişlik verilmesinin sebebi Bölüm 3.5: zaten kendi beş kareli Viterbi'si var, iki kez sıkılaştırmak gerçek hızlı geçkileri bastırıyor.

```
double offline_transition_width_cents(const PitchEngineId id) {
    return id == PitchEngineId::pitch_engine_v2 ? 250.0 : 150.0;
}
```

core/src/analysis_engine.cpp:987–989

PYTHON – YOL ARAMASI

```
# --- 2. karesel gecis cezali yol aramasi -----
def yolu_coz(iz, kat):
    if len(kat) < 3:
        return iz
    skor = [[math.log(max(s["puan"], 1e-6)) for s in kat[0]["secenekler"]]]
    geri = []
    for k in range(1, len(kat)):
        surekli = (iz[kat[k]["indeks"]]["zaman"] - iz[kat[k-1]["indeks"]]["zaman"]) <= BOLUM_BOSLUGU
        simdi, isaret = [], []
        for s in kat[k]["secenekler"]:
            en_iyi, secilen = float("-inf"), 0
            for j, evvel in enumerate(kat[k-1]["secenekler"]):
                gecis = 0.0
                if surekli:
                    d = sent(s["frekans"], evvel["frekans"]) / GENISLIK
                    gecis = -min(d * d, EN_COK_GECIS)
                toplam = skor[k-1][j] + gecis
                if toplam > en_iyi:
                    en_iyi, secilen = toplam, j
            simdi.append(en_iyi + math.log(max(s["puan"], 1e-6)))
            isaret.append(secilen)
        skor.append(simdi)
        geri.append(isaret)

    iyilestirilmis = [dict(k) for k in iz]
    imlec = max(range(len(skor[-1])), key=lambda i: skor[-1][i])
    for k in range(len(kat) - 1, -1, -1):
        secenek = kat[k]["secenekler"][imlec]
        iyilestirilmis[kat[k]["indeks"]]["frekans"] = secenek["frekans"]
        imlec = geri[k-1][imlec] if k > 0 else 0
    return iyilestirilmis
```

Bölüm 3.5'teki Viterbi ile aynı iskelet, üç farkla: geçiş karesel, 30 ms'den uzun aralıklı kareler **ayrı koşullar** sayılır (bir artikülasyon boşluğu üzerinden süreklilik varsayılmaz), ve arama yayınlanmış karelerin üzerinde çalışır — hop indeksleri üzerinde değil.

Neden ayrı bir arama yazılmış?

Dosyada zaten genel bir `resolve_track` çözücü var. Ama o, hop başına bir aday katmanı varsayar ve kare zamanlarını hop indeksinden hesaplar. Yayınlanmış bir iz ise **seyrek** — kapılardan geçmiş, köprülenmiş ve boşaltılmış kareler. Bu pas, tam olarak yayınlanmış karelerin üzerinde kendi aramasını koşar; zamanlarını ve seslendirme kararlarını olduğu gibi korur.

core/src/analysis_engine.cpp:1071-1078 (gerekçe yorumunda)

Seslendirme kararı bu pasta hiç yeniden ele alınmaz — yalnız frekanslar oynatılır. Sessiz bir kare sessiz kalır, seslendirilmiş bir kare seslendirilmiş kalır.

Deneyelim

PYTHON – TEK KARELİK OKTAV HATASI TAŞIYAN BİR İZ

```
hop = 0.0116
iz = [{"zaman": i * hop, "frekans": 440.0, "guven": 0.90} for i in range(8)]
iz[4]["frekans"], iz[4]["guven"] = 220.0, 0.85 # tek karelik oktav hatasi

def spektrum(indeks, f): # 440 her karede gercekten gur
    return 1.0 if abs(f - 440.0) < 5 else 0.02

print("once :", [round(k["frekans"]) for k in iz])
print("sonra:", [round(k["frekans"]) for k in yolu_coz(iz, katmanlar(iz, spektrum))])
```

ÇIKTI

```
once : [440, 440, 440, 440, 220, 440, 440, 440]
sonra: [440, 440, 440, 440, 440, 440, 440, 440]
```

Hatalı karenin kendi güveni 0,85 ile hiç de düşük değildi — nedensel motor onu makul bulmuştu. Yolun onu reddetme sebebi kendi zayıflığı değil, **komşularıyla uyumsuzluğu**: 440 alternatifi $0,90 \cdot 0,72 \cdot 1,0 = 0,65$ puanla masaya geliyor, buna karşılık 220'yi tutmak iki yönde $(1200/150)^2 = 64$ 'lük geçiş cezası ödüyor. Karşılaştırma yakın bile değil.

7.3 Başiboş nokta temizliği

İkinci pas daha basit bir soruyu sorar: "bu kısa koşu gerçekten bir nota mı, yoksa sessizliğin ortasında kalmış bir kaza mı?"

Koşul	Değer
Koşu uzunluğu	≤ 7 kare
Her iki yanında sessizlik	≥ 40 ms
Koşunun en yüksek güveni	$< 0,80$

Üçü birden sağlanırsa koşu atılır.

İki yandan yalıtılmış olma koşulu neden var?

Gerçek süslemeleri korumak için. Bir *çarpma*, süslediği notaya bitişiktir — sessizliğin ortasında yalnız kalmaz. Dolayısıyla bu testi hiçbir zaman geçemez ve kural onlara asla dokunmaz. Kuralı "kısa notaları at" diye okumak yanlış olur; doğru okuma "hiçbir şeye bağlanmayan kısa ve emin olmayan koşuları at"tır.

0,80 eşiği uydurulmuş değil: dinleyicinin işaretlediği her başiboş noktada **doğru davranıp sessiz kalan motor VPM-like'tı**, ve diğerlerinin yayınladığı koşular 0,66–0,81 güven aralığında tepe yapıyordu. Kısa ve yalıtılmış bir koşuyu VPM-like'ın kendi yayın barajına tutmak *kuraldır*; o karelere uydurulmuş bir sayı değil.

PYTHON – KOŞULARA AYIRMA VE ELEME

```
# --- 3. basibos kusu temizligi -----
def basibos_kosulari_at(iz, en_cok_kare=7, yalitim=0.040, guven_esigi=0.80):
    sesli = [i for i, k in enumerate(iz) if k["frekans"]]
    if len(sesli) < 2:
        return iz
    kosular, bas = [], 0
    for p in range(1, len(sesli) + 1):
        ayir = p == len(sesli) or iz[sesli[p]]["zaman"] - iz[sesli[p-1]]["zaman"] > 0.012
        if ayir:
            kosular.append((bas, p - 1)); bas = p
    at = set()
    for n, (ilk, son) in enumerate(kosular):
        if son - ilk + 1 > en_cok_kare:
            continue
        once = math.inf if n == 0 else iz[sesli[ilk]]["zaman"] - iz[sesli[kosular[n-1][1]]["zaman"]
        sonra = math.inf if n + 1 == len(kosular) else iz[sesli[kosular[n+1][0]]["zaman"] - iz[sesli[son]]
        ["zaman"]
        if once < yalitim or sonra < yalitim:
            continue
        if max(iz[sesli[p]]["guven"] for p in range(ilk, son + 1)) >= guven_esigi:
            continue
        at.update(sesli[p] for p in range(ilk, son + 1))
    return [k for i, k in enumerate(iz) if i not in at]
```

PYTHON – DENEME

```
iz2 = ([{"zaman": i*hop, "frekans": 440.0, "guven": 0.9} for i in range(6)]
        + [{"zaman": 0.30 + i*hop, "frekans": 300.0, "guven": 0.7} for i in range(3)]
        + [{"zaman": 0.60 + i*hop, "frekans": 440.0, "guven": 0.9} for i in range(6)])
print("once :", len(iz2), "kare")
print("sonra:", len(basibos_kosulari_at(iz2)), "kare")
```

ÇIKTI

```
once : 15 kare
sonra: 12 kare (yalitilmis 3 karelik 300 Hz kosusu atildi)
```

core/src/analysis_engine.cpp:1009–1020 (gerekçeli sabitler) · 1023–1059 (uygulama)

7.4 Nerede devreye giriyor

```
std::vector<EngineFrame> PitchEngine::analyse(const std::span<const float> samples, const double rate) {
    auto baseline = analyse_causal(samples, rate);
    if (profile_ != PitchEngineProfile::offline_track || baseline.empty()) return baseline;

    return drop_offline_stray_runs(
        refine_offline_harmonics(baseline, samples, rate, config_, id_)
    );
}
```

core/src/analysis_engine.cpp:1202–1212

Yani: **Çalma Modu** (canlı mikrofon) ham nedensel izi görür; **Dinleme/Çalışma Modu** (dosya analizi) bu iki pastan geçmiş hâlini görür. İki mod arasındaki fark artık yalnız gecikme değil, **kanıt miktarıdır**.

İki değişmez testlerle kilitli: `realtime` profili `analyse_causal` ile birebir aynı döner, ve nedensel geçişin taahhüt ettiği *gerçek* bir register sıçraması yol aramasından sağ çıkar.

core/tests/analysis_engine_tests.cpp

Temkinli davrandığının ölçülmüş bir kanıtı da var: farklı bir kayıta (`gercek-klarnet-calm`) dört motorun hiçbirinde tek kare bile değişmedi. Katman körü körüne düzeltirme yapmıyor; yalnız yolun açıkça karşı çıktığı yerlere dokunuyor.

`offline_track_v1` JSON'unda değişen kareler `change_reason: "offline_harmonic_path"` ile işaretlenir — yani hangi kareye dokunulduğu izlenebilir.

8. Karşılaştırma

Dördü de aynı soruyu soruyor ama farklı kanıtlara bakıyor. Bu bölüm, her birinin neyi nasıl yaptığını yan yana koyuyor — ve elimizdeki tek gerçek ölçümü.

8.1 Yapı taşları

	YIN v1	Pitch Engine V2	VPM-like	HAPT v1
Çekirdek fonksiyon	CMND (fark)	YIN + ACF + NSDF + spektral	Normalize ACF	NSDF
Aday sayısı	≤ 12	Dört kaynaktan, hepsi	1 (seçilmiş tepe)	Tüm NSDF tepeleri
Spektral kanıt	Tek frekans probu (oktav kurtarma + hayalet)	SWIPE' çekirdeği (tam FFT)	Akraba testi (1/3, 1/2, 1, 2, 3x)	Yarım/üçte-bir ızgara + tek doluluk
Faz kullanımı	—	—	—	Evet (IF kilidi)
Karar biçimi	Anlık, süreklilik cezalı	5 kare Viterbi	Anlık + veto/histerezis	Anlık, iki eşikli
Gecikme	0 kare	5 kare (≈53 ms)	0 kare	0 kare
Yayın barajı	0,55 (taze: 0,76)	0,70 / 0,55	0,80	0,55 / 0,32
Aşağı sıçrama onayı	3 kare (>900 sent)	harmonic_filter	3 kare (+2,5x vetosu)	2 kare
Hesap yükü	Düşük	Yüksek	Orta	Orta-yüksek

8.2 Aynı sorunlara verilen farklı cevaplar

"Bu aday gerçekten temel frekans mı, yoksa hayalet mi?"

YIN v1	2x ve 3x katlarının genişliğini kendi genişliğiyle kıyasla; oran 6'yı geçiyorsa kademeli ceza. Oktav terfisi için ayrıca teke harmonik doluluğu ara.
V2	Dört bağımsız tahminci ne diyor? İkisi 55 sent içinde anlaşıyorsa +0,20. Ayrıca SWIPE' asal harmonik çekirdeği.
VPM-like	Gerçek spektrumda 1/3, 1/2, 1, 2, 3x akrabalarını ölç; yerel tepe + mutlak + görel testleri geçeren pes olanı seç. Ama asla yukarı terfi ettirme.
HAPT	"Bir kat fazla pes olsaydım enerji nerede olurdu?" diye sor. Yarım/üçte-bir ızgarayı ölç. Pencere çözemiyorsa NSDF katlarına düş.

"Nota bitti mi, yoksa sadece kısıldı mı?"

YIN v1	2 ardışık %20 düşüş → bırakış. İki ayrı RMS tepesi (hızlı/yavaş sönen).
V2	Aynı desen, ama durum kare başına kuyruklanır — 5 kare geciken karar asla başka bir pencerenin kapısını uygulamaz.
VPM-like	Tam durum makinesi: release_suspected → aynı kontura mı döndü, farklı kontura mı? Ayrıca waiting_for_attack .
HAPT	İki tetikleyici: 2 keskin düşüş veya tepenin %35'inin altına inme (reverb kuyruğu). Tahminden bağımsız çalışır.

"Kısa bir kopma oldu, ne yapmalı?"

YIN v1 ve HAPT ortak [append_bridged](#) 'i kullanır (≤7 kare, iki uç ≤90 sent). V2 ve VPM-like kendi köprüleme mantıklarına sahiptir; V2 ayrıca **başlangıç köprüsü** ile notanın gerçek atak zamanını geriye dönük kurtarır.

8.3 Ölçülmüş sonuçlar

Ölçümün kaynağı

Şükrü Tunar uşşak taksimi, **85 aralık**, 8,7 saniye. Yer gerçeği bir **dinleyicinin hükmüdür** — pYIN değil, motorlar arası medyan değil. Kayıt: `data/annotations/sukru-tunar-ussak-taksim.verdicts.v1.json`. Sayılar `offline_track` profiline aittir.

Başlangıç durumu

Bu belgede anlatılan savunmaların hiçbiri yokken, dört motorun bu kayıttaki durumu:

Motor	Etiketli kusur	Oktav hatası
YIN v1	38	14
Pitch Engine V2	24	3
VPM-benzeri	46	18
Harmonik-Faz (HAPT)	50	12
Toplam oktav hatası: 47		

V2 daha o zaman en az oktav hatası yapan motordur (3). Viterbi'si olan tek motor o; Bölüm 6.2'nin tezini doğrudan doğruluyor.

Şu anki durum

Üç düzeltme turundan sonra — 120 Hz alt sınırı, YIN'in tek-harmonik koşulu, çevrimdışı yol iyileştirmesi ve merdivenin 5x'e uzatılması:

Motor	Etiketli kusur	Duruyor	Oktav hatası	Bozulan koruma
YIN v1	38	21	1	0
Pitch Engine V2	24	20	1	0
VPM-benzeri	46	32	4	0
Harmonik-Faz (HAPT)	50	33	1	0
Toplam oktav hatası: 47 → 7 · hiçbir motor hiçbir turda kötüleşmedi				

docs/TEST_BASELINE.md — 26 ve 31 Ağustos 2026 kayıtları

Okunacak birkaç şey var:

- YIN v1'in 14 → 1 düşüşü** büyük ölçüde tek-harmonik doluluk koşulundan geliyor: işaretli 13 oktav hatasının 12'si oktav kurtarma pasından kaynaklanıyordu.
- VPM-like'ın 18 → 4 düşüşü** büyük ölçüde 120 Hz alt sınırından geliyor — yani lag arama aralığının daralmasından, yeni bir kuraldan değil.
- "Duruyor" sütunu hâlâ yüksek.** Oktav *dışı* kusurlar (boşluk, kaçık nokta, zamanlama) çözülmedi; bu ölçüm turları oktav sorununa odaklıydı. VPM-like'ın 32 kusurunun çoğu **boşluktur** — sessiz kalması, yanlış söylemesi değil.
- Koruma hücreleri hiç bozulmadı.** Ölçüm kümesinde, motorların zaten doğru olduğu kareler ayrıca izleniyor; her turda bu sayı 0'da kaldı — düzeltmeler kazanç sağlarken başka yerde kaybettirmedi.

Bu tablo bir sıralama değildir

Tek bir kaydın 85 aralığı üzerinde, tek bir dinleyicinin hükümlerine göre yapılmış bir ölçüm. Farklı bir kayıta (`gercek-klarnet-calm`) çevrimdışı iyileştirme dört motorda da tek kare bile değıştirmede. Uygulamadaki motor seçimi de bir kalite sıralaması değildir; ilk açılıştaki YIN v1 seçimi yalnız geriye uyumluluk içindir.

8.4 Hangi motor neyi iyi yapıyor

Motor	Güçlü yanı	Bedeli
YIN v1	Basit, hızlı, öngörülebilir. Okunabilir referans uygulaması var. Sıfır gecikme.	Tek karelik kararlar; kanıtı erken çökertiyor.
Pitch Engine V2	Kanıt çeşitliliği + yol araması. Tek karelik hatalar yola oturmuyor. 900 Hz üstünde disiplinli.	≈53 ms gecikme, kare başına belirgin hesap (dört tahminci + FFT).
VPM-like	Sessiz kalması gerektiğinde sessiz kalıyor. En yüksek yayın barajı (0,80), en ayrıntılı atak/bırakış durum makinesi. Katlardan periyot rafinesi tiz seslerde kesin.	Muhafazakârlık boşluk kusuru olarak geri dönüyor; en çok etiketli kusur onda.
HAPT v1	Klarnet fiziğini doğrudan karar kuralına çeviriyor. Çözünürlük farkındalığı. Faz kilidiyle alt-sent doğruluk. İki eşikli histerezis "eksik kare" sorununu azaltıyor.	Gevşek sustain eşiği, sönen kuyrukları yakalamak için ayrı bir bırakış tespiti gerektiriyor.

Ek A — Parametreler

A.1 Ortak üretim yapılandırması

core/include/klarivision/core/analysis_engine.hpp:15–37

Alan	Değer	Anlamı
minimum_frequency_hz	120,0	En pes hipotez. $\tau_{\max}$ 'ı kısıtladığı için doğrudan bir doğruluk parametresidir.
maximum_frequency_hz	1500,0	En tiz hipotez $\rightarrow \tau_{\min}$
minimum_rms	0,015	Sessizlik kapısı (tüm motorlar için ortak)
window_size	1536	34,8 ms @44,1 kHz
hop_size	512	11,6 ms $\rightarrow \sim 86$ ölçüm/sn

A.2 Motora özel eşikler

Eşik	Değer	Nerede
YIN v1		
CMND aday eşiği	0,50	analysis_engine.cpp:122
Taze aday barajı	0,76	analysis_engine.cpp:240
Aday değerlendirme tabanı	0,55	analysis_engine.cpp:249
Süreklilik cezası	0,30 / 700 sent	analysis_engine.cpp:253
Hayalet oran eşiği	6,0	harmonic_arbitration.cpp:74
Oktav kurtarma baskınlığı	8×, >800 Hz	analysis_engine.cpp:159–171
Aşağı sıçrama	>900 sent, 3 kare	analysis_engine.cpp:558–573
Yayın/köprü tabanı	0,55	analysis_engine.cpp:933
Pitch Engine V2		
YIN adayı eleme	0,50	v2_session.cpp:313
Uzlaşma penceresi	55 sent	v2_session.cpp:371
Emisyon ağırlıkları	0,40 / 0,20 / 0,40	v2_session.cpp:379
Spektral aday tabanı	0,90	v2_session.cpp:387
Viterbi gecikmesi	5 kare	analysis_engine.cpp:420 (kurucu)
Geçiş genişliği	700 sent	fixed_lag_tracker.hpp:35
Oktav ek ücreti	+0,12 (≥ 850 sent)	fixed_lag_tracker.cpp:30
Yayın barajı	0,70 / 0,55	v2_session.cpp:490–495
VPM-like		
En az periyodiklik	0,38	vpm_like.hpp:52
Çıkış güveni	0,80	vpm_like.hpp:53
"En güçlüye yakın" oranı	0,90	vpm_like.hpp:54
Görelî spektral destek	0,080	vpm_like.hpp:55
Mutlak spektral destek	0,005	vpm_like.hpp:56
En yüksek periyot katı	6	vpm_like.hpp:57
Sıçrama tanımı	≥ 650 sent, ± 110 sent	vpm_like.cpp:23–33
Üst hat vetosu	2,5×	vpm_like.cpp:124

HAPT v1		
Harmonik sayısı	5	hapt.hpp:38
Tek harmonik eşiği	0,10	hapt.hpp:39
IF kilidi periyodikliği	0,75	hapt.hpp:42
IF kilidi düzeltme sınırı	40 sent	hapt.hpp:43
IF kilidi genlik sınırı	6 dB	hapt.hpp:44
Onset / sustain eşiği	0,55 / 0,32	hapt.hpp:50–51
Sustain kontur mesafesi	180 sent	hapt.hpp:52
Sıçrama toleransı / onay	30 sent / 2 kare	hapt.hpp:55–56
Bırakış oranları	0,80 / 0,35 / 0,40	hapt.hpp:67–69
Çevrimdışı katman		
Desteksiz alternatif tabanı	0,30	analysis_engine.cpp:997
Alternatif iskontosu	0,72	analysis_engine.cpp:1000
Bölüm boşluğu	30 ms	analysis_engine.cpp:1003
En yüksek geçiş cezası	12,0	analysis_engine.cpp:1007
Geçiş genişliği	150 / 250 sent	analysis_engine.cpp:987–989
Başıboş koşu	≤7 kare, 40 ms, 0,80	analysis_engine.cpp:1011–1020

Ek B — Dosya haritası

Dosya	İçeriği
Ortak / oturum	
<code>core/src/analysis_engine.cpp</code>	Üretim oturumu. YIN v1'in tamamı, VPM ve HAPT'ın oturum katmanı, köprüleme, çevrimdışı iyileştirme.
<code>core/include/.../analysis_engine.hpp</code>	<code>PitchEngineConfig</code> , <code>EngineFrame</code> , motor kimlikleri.
<code>core/src/harmonic_probe.cpp</code>	Hann pencereli tek frekans DFT probu; ana lob genişliği.
<code>core/src/harmonic_arbitration.cpp</code>	<code>spectral_existence</code> — hayalet alt-harmonik tespiti.
YIN v1	
<code>src/klarivision/pitch/yin.py</code>	Okunabilir referans uygulaması (108 satır).
<code>core/src/analysis_engine.cpp:86–176</code>	<code>yin_candidates</code> — CMND + oktav kurtarma.
<code>core/src/analysis_engine.cpp:194–272</code>	<code>causal_yin_choice</code> — hayalet + süreklilik puanlaması.
<code>macos/.../LivePitchAnalyzer.swift:3772</code>	vDSP hızlandırmalı canlı kopya.
Pitch Engine V2	
<code>core/src/pitch_engine_v2_session.cpp</code>	Aday üretimi (4 kaynak), puanlama, yayın kapıları.
<code>core/src/fixed_lag_tracker.cpp</code>	Sabit gecikmeli Viterbi.
<code>core/src/swipe_prime.cpp</code>	FFT + SWIPE' asal harmonik çekirdeği.
<code>core/src/mpm.cpp</code>	NSDF / McLeod aday üretici (referans sürüm).
<code>core/src/pitch_engine_v2.cpp</code>	Sade seçim sınırı (<code>select_candidate</code>).
VPM-like	
<code>core/src/vpm_like.cpp</code>	ACF, kat rafinesi, spektral düzeltme, <code>VPMLikeTracker</code> .
<code>core/include/.../vpm_like.hpp</code>	Yapılandırma ve tanı yapıları.
<code>docs/VPMLikePitchEngine.md</code>	Yöntemin türetimi ve kaynakları.
HAPT v1	
<code>core/src/hapt.cpp</code>	NSDF, harmonikler-arası veto, IF kilidi, <code>HAPTTracker</code> .
<code>core/include/.../hapt.hpp</code>	Yapılandırma; her eşliğin gerekçesi yorumda.
<code>docs/HAPTPitchEngine.md</code>	Tam türetim, $\pm 93,75$ Hz aralığı, bilinen sınırlar.
pYIN ve ölçüm	
<code>src/klarivision/pitch/pyin.py</code>	librosa.pyin adaptörü (çevrimdışı referans).
<code>scripts/quick_pitch_check.py</code>	Motorları dinleyici hükümlerine karşı ~14 s'de puanlar.
<code>scripts/pitch_engine_divergence_map.py</code>	Motorların ayrıştığı yerleri bulur, inceleme sayfasını üretir.
<code>docs/TEST_BASELINE.md</code>	Ölçüm kayıtları ve gerekçeler.

Bu belge 31 Ağustos 2026 tarihli çalışma ağacına göre yazıldı: [5a39822](#) commit'i artı o gün yapılan, henüz commit'lenmemiş çevrimdışı harmonik merdiven değişikliği (Bölüm 7.2). Satır numaraları o duruma aittir; kod değişikçe kaymaları beklenir — her alıntının yanındaki fonksiyon/sabit adı kalıcı çapadır.

Ek C — Python örnekleri hakkında

Bu belgedeki Python kod bloklarının tamamı tek bir dosyada toplanmıştır:

docs/perde-motorlari-ornekler.py

Dosya **hiçbir dış bağımlılık kullanmaz** — yalnız standart kütüphanenin `math` modülü. Herhangi bir Python 3 kurulumunda doğrudan çalışır:

```
python3 docs/perde-motorlari-ornekler.py
```

Belgedeki her "çıkıtı" bloğu, bu dosyanın gerçekten ürettiği çıktıdır; elle yazılmamıştır.

Ne olduğu — ve ne olmadığı

Bu kod üretim kodu değildir

Örnekler **öğretme amaçlıdır**: gerçek motorların karar mantığını, okunabilirliği önceleyen sade bir biçimde yeniden yazarlar. Üretimdeki C++/Swift kodunun yerine geçmezler ve onunla birebir aynı sonucu vermezler.

Kasten dışarıda bırakılanlar: vektör hızlandırması (vDSP), FFT tabanlı SWIPE' çekirdeği, tam pencereleme/DC işleme, tanı kayıtları, ve gerçek spektral prob (örneklerde yerine basit bir sözlük veya fonksiyon konur). Eşikler ve formüller ise **gerçek değerlerdir** — hangi sayının nereden geldiğini görmek için kullanılabilirler.

Hangi örnek nerede

Bölüm	Konu	Fonksiyonlar
2.1–2.3	YIN aday üretimi	<code>klarnet</code> , <code>fark_fonksiyonu</code> , <code>cmdn</code> , <code>parabolik</code> , <code>yin_adaylari</code>
2.6	YIN seçim merdiveni	<code>hayalet_orani</code> , <code>hayalet_cezasi</code> , <code>nedensel_secim</code> , <code>SicramaMuhafizi</code>
3.4–3.5	V2 emisyon + Viterbi	<code>emisyon_puanlari</code> , <code>gecis</code> , <code>viterbi_coz</code> , <code>SabitGecikmeliIzleyici</code>
4.2–4.5	VPM-like seçim	<code>kabul_edilen_lag</code> , <code>kat_rafinesi</code> , <code>spektral_duzeltme</code> , <code>VPMIzleyici</code>
5.2, 5.5	HAPT veto ve histerezis	<code>izgara_vetosu</code> , <code>tek_harmonik_dolulugu</code> , <code>hapt_puanla</code> , <code>yayinlanabilir</code>
7.2–7.3	Çevrimdışı katman	<code>katmanlar</code> , <code>yolu_coz</code> , <code>basibos_kosulari_at</code>

Denemeye başlamak için üç fikir

- Eşiği kırın.** `yin_adaylari`'nin `esik` değerini 0,50'den 0,15'e indirin. Aday listesi ne kadar kısalıyor? Doğru cevap hâlâ orada mı?
- Klarneti keman yapın.** `klarnet` fonksiyonundaki genlikleri değiştirip temel frekansı gür, harmonikleri zayıf yapın (0,30 ↔ 1,00). HAPT'ın tek harmonik doluluğu ne oluyor? Örnekteki 220 Hz hayaleti hâlâ eleniyor mu?
- Viterbi'yi zorlayın.** `t3` örneğindeki bozuk kareyi *iki* ardışık kareye çıkarın. Yol hangi noktada 220 Hz'i kabul etmeye başlıyor? Bu, "yalıtılmış hata" ile "gerçek register değişimi" arasındaki sınırın nerede olduğunu doğrudan gösterir.

Doğrulama

Belge derlenirken her kod bloğu bu dosyadan okunur ve her çıktı bloğu dosya çalıştırılarak üretilir. Yani belgedeki bir kod parçası bozulursa, çıktı da bozulur — ikisi birbirinden bağımsız olarak eskiyebilir.